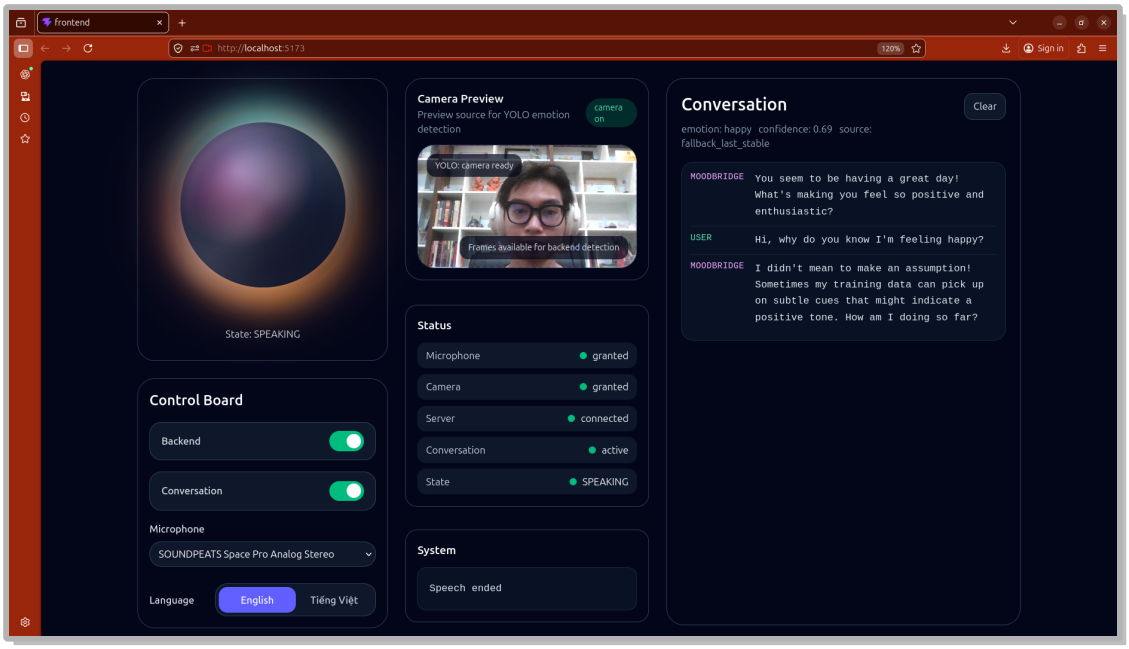
MoodBridge: Technical Report

A Real-Time Emotion-Aware Conversational AI System

Dacekey | Le Khoi Nguyen

April 17, 2026



Department of Artificial Intelligence
FPT University Ho Chi Minh City

Contents

Abstract **7**

1 Introduction **8**
1.1 Project Motivation 8
1.2 Core Objective 8
1.3 Scope of This Report 8

2 System Vision and Design Philosophy **10**
2.1 High-Level Product Vision 10
2.2 Guiding Engineering Principles 10
2.3 Why the Architecture Changed Over Time 11

3 Final System Overview **12**
3.1 Operational Summary 12
3.2 Final High-Level Architecture 12
3.3 Core Interaction Loop 13

4 Final Architecture Distillation **14**
4.1 Frontend Responsibilities 14
4.2 Backend Responsibilities 14
4.3 Why Backend-First Control Was the Right Choice 15

5 System Components **16**
5.1 Backend Structure 16
5.1.1 `conversation_ws.py` 16
5.1.2 `conversation_runtime.py` 17
5.1.3 `voice_input_controller.py` 17
5.1.4 Service Modules 18
5.2 Frontend Structure 18
5.2.1 Pages Layer 19
5.2.2 Components Layer 19
5.2.3 Library Layer 23
5.2.4 Types Layer 23

6 Emotion Detection Pipeline **25**
6.1 Role of Emotion Detection in the Final Product 25
6.2 Model and Inference Context 25
6.3 Bootstrap Detection Procedure 25
6.4 Why Short Detection Was Better Than Continuous Detection 26
6.5 Multi-Frame Voting 26
6.6 Output Usage 26

7 Realtime Audio Pipeline **27**
7.1 Transition from File Upload to PCM Streaming 27
7.2 Audio Format 27
7.3 Why PCM Streaming Was Necessary 27
7.4 Microphone Lifecycle 28

8 Backend Voice Activity Detection and Utterance Segmentation **29**
8.1 VAD Strategy 29
8.2 Core Segmentation Logic 29
8.3 Important Heuristics 29
8.4 Why Backend VAD Was the Right Choice 30
8.5 Why Barge-In Was Not Supported in the 1st Version 30

9 Speech-to-Text Integration **32**
9.1 Role of STT 32
9.2 Technologies Used in Speech Processing 32
9.3 Transcript Filtering 33
9.4 Practical Importance 33

10 LLM Integration and Prompting **34**
10.1 Conversation Service Role 34
10.2 Prompt Philosophy 34
10.3 Opening Message Generation 34
10.4 Observed LLM Failure Mode 35
10.5 Technologies Used and LLM Model Selection 35
10.5.1 Model Selection 35
10.6 Why an Open-Source Model Was Used Instead of a Closed API Model . 36

11 TTS Integration **37**
11.1 Role of TTS 37
11.2 Streaming-Oriented Design 37
11.3 Listening Suppression During Speaking 37
11.4 Technologies Used in Text-to-Speech 37

12 Conversation State Machine **39**
12.1 Importance of the State Machine . . . 39
12.2 Core States . . . 39
12.3 Representative Transition Flow . . . 40
12.4 Why `ARMED_LISTENING` Matters . . . 40
12.5 Design Rationale for the Conversation State Machine . . . 40
12.6 State Design Trade-Offs . . . 42

13 WebSocket Protocol Design **43**
13.1 Event-Driven Messaging Model . . . 43
13.2 Session and Conversation Control Events . . . 43
13.3 Audio Streaming Events . . . 44
13.4 Emotion Bootstrap Events . . . 44
13.5 Backend-to-Frontend Response Events . . . 45
13.6 Connection and Health Monitoring Events . . . 45
13.7 Error and Exception Handling Events . . . 46
13.8 Conversation Lifecycle Events . . . 46
13.9 Why Explicit Events Matter . . . 46

14 Final End-to-End User Flow **47**
14.1 Setup and Permissions . . . 47
14.2 Conversation Bootstrap . . . 47
14.3 Normal Dialogue Loop . . . 47

15 Key Engineering Challenges and Debugging Outcomes **49**
15.1 Challenge 1: Camera Flickering and Missing Frames . . . 49
15.1.1 Resolution . . . 49
15.2 Challenge 2: Emotion Detection Triggering at the Wrong Time . . . 49
15.2.1 Resolution . . . 50
15.3 Challenge 3: Microphone Selection and Readiness . . . 50
15.3.1 Resolution . . . 50
15.4 Challenge 4: Auto-Stop and Sudden Disconnects . . . 50
15.4.1 Resolution . . . 50
15.5 Challenge 5: False Speech Capture After Listening . . . 51
15.5.1 Resolution Status . . . 51
15.6 Challenge 6: Local LLM Timeout During Opening . . . 51
15.6.1 Resolution Status . . . 52
15.7 Challenge 7: Missed User Speech After Entering `ARMED_LISTENING` . . . 52
15.7.1 Resolution . . . 52

16 UI and UX Stabilization **53**
16.1 Conversation UI Evolution . . . 53
16.2 Orb Behavior and State Synchronization . . . 53
16.3 Session Visibility and Control Flow . . . 54
16.4 System Status and Observability . . . 55
16.5 Key Practical Improvements . . . 55

17 Deployment and Operational Notes **56**
17.1 Environment Setup Strategy . . . 56
17.2 CLI and Script Workflow . . . 56
17.3 Practical Development Command . . . 56

18 Current System Capabilities **57**
18.1 What the Final System Can Do . . . 57
18.2 Conversation Flow Stability . . . 58
18.3 Operational Constraints . . . 59
18.4 System Reliability Features . . . 59
18.5 System Maturity Assessment . . . 60

19 Known Limitations **61**
19.1 Behavioral Limitations . . . 61
19.2 Technical Limitations . . . 61
19.3 Operational Limitations . . . 62
19.4 Feature Limitations . . . 63
19.5 Design Trade-Off Summary . . . 63

20 Future Work **64**
20.1 Short-Term Priorities . . . 64
20.2 Medium-Term Priorities . . . 65
20.3 Long-Term Directions . . . 65
20.4 Application-Oriented Expansion . . . 66
20.5 Strategic Vision . . . 67

21 System Technical Stack **68**
21.0.1 Frontend Technologies . . . 68
21.0.2 Backend Technologies . . . 69
21.0.3 Artificial Intelligence and Machine Learning Components . . . 70
21.0.4 Audio and Communication Technologies . . . 70
21.0.5 System and Runtime Environment . . . 71
21.0.6 Summary . . . 71

22 Conclusion **72**

A MoodBridge System Diagram **74**
A.1 System Architecture Diagram . . . 76
A.2 Conversation State Machine Diagram . . . 77
A.3 Conversation Interaction Sequence . . . 78
A.4 Emotion Bootstrap Sequence . . . 79
A.5 WebSocket Event Flow . . . 79
A.6 Speech Processing Pipeline . . . 80
A.7 Emotion Detection Pipeline . . . 81
A.8 TTS Response Pipeline . . . 82
A.9 Session Lifecycle Diagram . . . 83
A.10 Frontend UI Component Structure . . . 84
A.11 Service Dependency Diagram . . . 85
A.12 Runtime Error Handling Flow . . . 86
A.13 Microphone Initialization Timing Fix . . . 86

B MoodBridge System Configuration **87**
B.1 Audio Processing Configuration . . . 87
B.2 Network Port Configuration . . . 87

C MoodBridge Startup Commands **88**
C.1 System Prerequisites . . . 88
C.1.1 Required Software . . . 88
C.1.2 Recommended Python Packages . . . 89
C.1.3 Recommended System Utilities . . . 89
C.2 Option 1: Manual Startup Using Two Terminals . . . 89
C.2.1 Terminal 1: Backend Runtime . . . 90
C.2.2 Terminal 2: Frontend Interface . . . 90
C.3 Option 2: Script-Based Startup . . . 90
C.3.1 Start the Full System . . . 90
C.3.2 Stop the System . . . 91
C.4 Demo Conversation Startup . . . 91
C.5 Environment Initialization . . . 91
C.5.1 Load Environment Variables . . . 92
C.6 Verification Commands . . . 92
C.6.1 Check Backend Availability . . . 92
C.6.2 Check Active Network Ports . . . 92
C.6.3 Check Frontend Server . . . 92
C.7 Common Troubleshooting Scenarios . . . 92
C.7.1 Frontend Does Not Start . . . 92
C.7.2 Backend Fails to Start . . . 93
C.7.3 Microphone or Speaker Not Detected . . . 93

C.8 Engineering Recommendation 94

D Hardware and Software Requirements 95

D.1 Minimum Hardware Requirements 95

D.2 Recommended Hardware 95

D.3 Software Requirements 95

E Diagnostic and Development Tools 96

E.1 Audio Device Diagnostic Tool 96

E.1.1 Script 96

E.1.2 Purpose 96

E.1.3 Functionality 97

E.1.4 Typical Usage 97

E.1.5 Future Improvement 97

E.2 Text-to-Speech Verification Tool 98

E.2.1 Script 98

E.2.2 Purpose 98

E.2.3 Functionality 98

E.2.4 Typical Usage 98

E.2.5 Listing Available Voices 99

E.3 Language Model Connectivity Test Tool 99

E.3.1 Script 99

E.3.2 Purpose 99

E.3.3 Functionality 99

E.3.4 Typical Usage 100

E.4 Engineering Role of Diagnostic Tools 100

F Project Documentation and Supporting Directories 101

F.1 Repository-Level Supporting Materials 101

F.2 Project Documentation Resources 101

F.2.1 `docs/` 101

F.2.2 `docs/diagram/mermaid/` 102

F.2.3 `docs/diagram/ASCII/` 102

F.3 Supporting Directory Descriptions 102

F.3.1 `demos/` 102

F.3.2 `docs/` 102

F.3.3 `env/` 103

F.3.4 `images/` 103

F.3.5 `models/` 103

F.3.6 `scripts/` 103

F.4 How Readers Can Use These Materials 104

Abstract

MoodBridge is a real-time multimodal conversational AI system designed to initiate and maintain natural spoken interaction by combining facial emotion detection, speech processing, large language model inference, text-to-speech synthesis, and a WebSocket-based frontend–backend architecture. Over the course of the project, the product architecture evolved significantly: the system started as a set of local experimental demos and gradually matured into a browser-based, backend-orchestrated conversation platform. This report consolidates the final engineering direction from the full set of project notes and intermediate summaries, while deliberately filtering out superseded plans and unstable early-stage assumptions.

The final system is centered around a backend-owned conversation runtime. The browser frontend is responsible mainly for user interface, camera preview, microphone capture, and lightweight event transport. The backend controls the state machine, voice activity detection, transcript filtering, opening-message generation, response orchestration, and safe lifecycle management of the session. A short camera-driven emotion detection phase at the beginning of the session provides soft emotional context for the opening message. After that, the system transitions into normal audio conversation mode, where speech is streamed as PCM frames over WebSocket, segmented with backend voice activity detection, transcribed with Whisper, processed by an LLM, and rendered through TTS.

This document presents the project rationale, final architecture, major implementation details, protocol design, state machine, engineering trade-offs, problems encountered during development, debugging outcomes, current system capabilities, limitations, and recommended future improvements.

Beyond its immediate technical contributions, the system represents an early foundation toward building intelligent interactive agents capable of perceiving and responding to human emotional signals. By combining perception, dialogue, and adaptive behavior within a unified realtime runtime, the project moves incrementally toward systems that can better understand human states through emotion and interaction. In the long term, this direction aims to narrow the functional and experiential gap between human users and robotic or software assistants, enabling more natural, empathetic, and human-like interaction.

Chapter 1

Introduction

1.1 Project Motivation

Human conversation is not purely textual. It is contextual, temporal, multimodal, and sensitive to subtle cues such as tone, facial expression, speaking rhythm, and turn-taking. Conventional chatbots typically begin with generic prompts and interact in a rigid request–response manner. MoodBridge was conceived as a personal multimodal AI project to explore a more natural interaction pattern: an AI assistant that can observe an initial facial expression, start the conversation with a gentle context-aware opener, and then continue in real-time voice dialogue.

The project is explicitly designed around an important ethical boundary: emotion estimation is treated as a soft contextual cue rather than a diagnostic or authoritative statement. This principle shaped both the prompt design and the UX. The goal is not to claim emotional certainty but to make the opening interaction feel more human, less mechanical, and more situationally aware.

1.2 Core Objective

The final core objective achieved by MoodBridge can be stated as follows:

> Build a real-time conversational AI web application that uses short camera-based facial emotion detection as contextual input for conversation opening, then supports continuous voice interaction through streaming speech capture, backend-controlled state transitions, LLM response generation, and TTS output.

1.3 Scope of This Report

This report is not a chronological diary. Instead, it is a distilled technical consolidation of the final engineering state of the project. It emphasizes:

- the final product architecture rather than every intermediate design,
- the stable interaction model rather than deprecated experiments,
- the actual system responsibilities of frontend and backend,
- major engineering lessons learned from development and debugging,
- the current completion status and future roadmap.

Chapter 2

System Vision and Design Philosophy

2.1 High-Level Product Vision

MoodBridge is positioned as a lightweight multimodal AI web application with the following characteristics:

- real-time interaction instead of batch-style processing,
- browser-based user access,
- camera and microphone permission-driven session startup,
- emotionally aware but uncertainty-conscious opening behavior,
- voice-first conversation loop,
- modular backend services,
- architecture suitable for iterative improvement and eventual deployment.

2.2 Guiding Engineering Principles

Across the project documents, several engineering priorities remained consistent even when the implementation changed:

1. **Stability before polish.** The system should work reliably enough for demonstration and iteration before advanced UX or animation details are optimized.
2. **Backend ownership of core logic.** State transitions, utterance decisions, transcript validation, and AI orchestration should reside in the backend rather than being fragmented across the frontend.
3. **Modularity.** Emotion detection, speech recognition, TTS, conversation generation, and runtime coordination should remain separable services.

4. **Real-time responsiveness.** The system must support streaming-oriented interaction and avoid long blocking flows where possible.

5. **Ethical phrasing.** Emotion-related language must remain probabilistic and gentle.

2.3 Why the Architecture Changed Over Time

The earliest project notes and roadmap documents proposed a fairly broad architecture, including dashboard ideas, REST endpoints, future deployment layers, and multiple phases of development. As implementation progressed, practical integration work revealed several important realities:

- realtime interaction is fundamentally easier to manage with a WebSocket-first design,
- compressed browser audio formats are inconvenient for robust VAD,
- frontend-heavy conversation orchestration becomes fragile quickly,
- audio timing and session keepalive are often bigger sources of instability than model inference,
- camera-based emotion detection is most useful as a short bootstrap phase rather than a continuous loop in the current demo scope.

These realizations led to the final architecture documented in later work summaries.

Chapter 3

Final System Overview

3.1 Operational Summary

The final MoodBridge system operates in two main phases:

1. **Emotion bootstrap phase.** The frontend opens the camera preview, sends a small number of sampled frames to the backend, and the backend performs multi-frame emotion inference using a YOLO-based model. The backend votes over the predictions and returns a final emotion estimate.

2. **Realtime voice conversation phase.** The frontend starts the conversation after the emotion result is available. The backend uses that result as context for generating an opening message, plays the opening through TTS, transitions into a listening-ready state, and then handles voice interaction using PCM audio streaming, VAD, STT, LLM generation, and TTS playback.

3.2 Final High-Level Architecture

```
Browser Frontend
    - Camera preview
    - Microphone capture
    - WebSocket client
    - State and message rendering
    - Lightweight control orchestration

            |
            | WebSocket events (JSON + base64 media payloads)
            v

FastAPI Backend
```

```
- WebSocket endpoint
- Conversation runtime
- Voice input controller / VAD
- Emotion service
- Speech-to-text service
- Conversation / LLM service
- Text-to-speech service
```

3.3 Core Interaction Loop

The end-to-end interaction pattern can be summarized as:

```
Camera -> Emotion Detection -> Opening Context
Microphone -> PCM Streaming -> Backend VAD -> STT
STT + Emotion + History -> LLM -> TTS -> Audio Output
```

Chapter 4

Final Architecture Distillation

4.1 Frontend Responsibilities

In the final system, the frontend is intentionally lightweight relative to the backend. Its main responsibilities are:

- establish and maintain a WebSocket connection,
- display UI state, conversation messages, and system status,
- obtain camera and microphone permissions,
- show camera preview,
- capture frames from the preview during emotion bootstrap,
- encode and stream audio PCM frames during listening phases,
- send control events such as session start, conversation start, and pings,
- stop or restart microphone capture based on backend state updates.

The frontend is *not* the authority for conversation decisions. It reacts to backend-driven state rather than inventing its own dialogue flow.

4.2 Backend Responsibilities

The backend is the authoritative orchestrator. Its responsibilities include:

- session lifecycle management,
- conversation state machine control,
- safe WebSocket send behavior,

- emotion inference and voting,
- audio stream decoding,
- voice activity detection and utterance segmentation,
- STT invocation,
- transcript filtering,
- LLM prompt construction and response generation,
- TTS invocation and playback coordination,
- handling disconnects and re-arming the listening state where appropriate.

4.3 Why Backend-First Control Was the Right Choice

This architecture emerged as the practical answer to several earlier problems:

- frontend state duplication was causing fragile behavior,
- speech timing needs a single source of truth,
- reconnects and disconnects are easier to reason about when the backend owns the runtime,
- transcript quality filtering should occur close to the STT output,
- emotion context is best stored in runtime state rather than passed ad hoc through UI callbacks.

Chapter 5

System Components

5.1 Backend Structure

The core backend structure stabilized around the following conceptual modules:

```
backend/
  app/
    main.py
    ws/
      conversation_ws.py
    runtime/
      conversation_runtime.py
      voice_input_controller.py
      runtime_services.py
    state/
      conversation_state.py
    schemas/
      ws_messages.py
  services/
    emotion_service.py
    speech_service.py
    conversation_service.py
    tts_service.py
```

5.1.1 `conversation_ws.py`

This module hosts the primary WebSocket endpoint and is responsible for transport-layer coordination. Its duties include:

- accepting client sessions,

- routing incoming event messages,
- maintaining safe send behavior,
- bridging audio frames into the voice input controller,
- triggering STT and feeding cleaned transcripts into the runtime,
- receiving emotion detection frames and returning final emotion results.

5.1.2 `conversation_runtime.py`

This module is the core orchestrator of the entire session. It manages:

- conversation states,
- transitions between opening, thinking, speaking, and listening,
- opening-message generation,
- use of the latest detected emotion as prompt context,
- runtime memory such as recent turns,
- safe state publication to the frontend.

5.1.3 `voice_input_controller.py`

This component handles lower-level speech input concerns:

- PCM frame ingestion,
- buffer management,
- VAD decisions,
- warmup windows,
- post-utterance cooldown,
- minimum speech commitment thresholds,
- temporary WAV output for STT.

5.1.4 Service Modules

The service layer separates core capabilities:

Emotion Service
loads the **YOLO**-based model, decodes input frames, runs prediction, and performs multi-frame voting.

Speech Service
provides speech recognition support, primarily through **Whisper**-based transcription.

Conversation Service
builds prompts and communicates with the large language model (LLM), in MoodBridge case, is **Llama3** through Ollama. This is an open-source LLM model.

TTS Service
converts textual response output into synthesized speech and coordinates playback state.

5.2 Frontend Structure

The frontend stabilized around the following important files:

```
frontend/
  src/
    pages/
      ConversationPage.tsx
    components/
      camera/ CameraPreviewCard.tsx
      conversation/ ConversationStream.tsx
      ui/ ToggleSwitch.tsx
      orb/ VoiceOrb.tsx
      setup/
        LanguageSwitch.tsx
        StatusPanel.tsx
        SystemPanel.tsx
        VoiceSessionPanel.tsx
    lib/
      pcmRecorder.ts
    types/
      ws.ts
```

5.2.1 Pages Layer

`ConversationPage.tsx`

This file serves as the primary orchestration layer in the browser. It coordinates the lifecycle of the conversation session and acts as the central integration point between the UI, WebSocket communication, and hardware interaction.

It manages:

- WebSocket lifecycle
- handling of incoming state updates
- emotion bootstrap sequencing
- microphone recorder lifecycle
- ping heartbeat mechanism
- coordination of when PCM streaming should or should not be active

Architecturally, this component behaves similarly to a client-side runtime controller, mirroring the backend conversation runtime in structure and responsibility.

It ensures that:

- audio is only streamed when the system is ready
- state transitions are synchronized with backend signals
- hardware resources are initialized safely
- session lifecycle remains deterministic

5.2.2 Components Layer

The components folder contains reusable UI modules responsible for rendering the interface and presenting system state to the user. These components are intentionally separated from orchestration logic to improve maintainability and readability.

Camera Components

`CameraPreviewCard.tsx` This component exposes the live video element and displays camera preview status.

A key implementation detail was ensuring that the camera setup effect depends only on stable state variables rather than transient inline callbacks. Earlier versions of the system experienced repeated camera teardown and restart cycles due to unstable dependency references in React effects.

Stabilizing this behavior significantly improved runtime reliability and prevented unnecessary hardware reinitialization.

Responsibilities:

- initialize camera stream
- display video preview
- report camera readiness state
- prevent repeated camera restart loops

Conversation Components

`ConversationStream.tsx` This component renders the visible dialogue history between the user and the assistant.

Its main purpose is to maintain conversational clarity during live interaction while separating conversational content from lower-level system diagnostics.

The conversation stream was later simplified to focus on readable dialogue turns rather than debug output. A presentation refinement also changed the assistant label from a generic system identifier to the product-facing identity:

```
MOODBRIDGE
```

Responsibilities:

- render conversation messages
- display user and assistant dialogue
- maintain readable conversation flow
- separate system logs from user dialogue

Orb Components

`VoiceOrb.tsx` This component represents the visual identity of the assistant and acts as a conversational presence indicator.

The orb is designed to visually reflect the current interaction phase, such as:

- listening
- thinking
- speaking

During development, more complex animation behavior was explored. However, stability and clarity were prioritized over visual complexity, resulting in a lightweight presentation component rather than a logic-heavy interface.

Responsibilities:

- display assistant presence
- reflect conversation state visually
- provide user feedback during interaction
- improve user engagement

Setup Components

These components configure the conversation environment and expose system status information.

`LanguageSwitch.tsx` This component allows the user to select the conversation language, typically between English and Vietnamese.

Language selection directly affects:

- speech recognition configuration
- text-to-speech voice selection
- language model prompting behavior

Responsibilities:

- select conversation language
- update session configuration
- improve multilingual usability

`StatusPanel.tsx` This component displays the current operational state of the system in a concise monitoring view.

It provides visibility into:

- connection status
- conversation state
- microphone readiness
- camera readiness
- speaking activity

The primary engineering value of this component is observability during runtime.

`SystemPanel.tsx` This component displays system-level runtime messages separately from the user-facing conversation interface.

Separating system feedback from conversational content improved usability and debugging efficiency.

Responsibilities:

- display system messages
- show runtime diagnostics
- separate system logs from dialogue

`VoiceSessionPanel.tsx` This component manages controls related to voice interaction and microphone configuration.

A critical addition to this component was the microphone device selector. Once the system transitioned to browser-based PCM streaming, selecting the correct input device became essential for stable operation.

Responsibilities:

- manage session controls
- display microphone state
- allow microphone selection
- connect hardware input to recorder pipeline

UI Components

`ToggleSwitch.tsx` This is a reusable UI primitive used to represent binary interaction states.

Although it does not directly affect the AI pipeline, it improves interface consistency and reduces duplicated UI logic across components.

Responsibilities:

- provide standardized toggle behavior
- improve UI consistency
- support reusable interface components

5.2.3 Library Layer

`pcmRecorder.ts`

The browser PCM recorder became central after the system transitioned away from MediaRecorder to support low-latency streaming audio.

This component is responsible for converting raw microphone input into a format compatible with the backend speech processing pipeline.

Responsibilities:

- microphone capture
- downsampling to required sample rate
- converting audio to mono int16 format
- chunking audio into time-aligned frames
- base64 encoding audio frames
- streaming audio over WebSocket
- selecting input device via deviceId

This module forms the foundation of real-time speech streaming in the browser.

5.2.4 Types Layer

`ws.ts`

This file defines the frontend-side TypeScript types for WebSocket communication.

It acts as the protocol contract between the browser and the backend system, ensuring consistent message structure across the application.

Responsibilities:

- define WebSocket message schemas
- standardize communication format
- reduce runtime errors
- support protocol evolution

Message categories include:

- session control messages
- conversation state updates

- emotion detection events
- audio streaming events
- transcript messages
- response messages
- error notifications
- heartbeat signals

Chapter 6

Emotion Detection Pipeline

6.1 Role of Emotion Detection in the Final Product

Although early project descriptions envisioned broader or continuous emotional awareness, the final design narrowed this component to a focused and practical role:

> Use camera-based emotion detection primarily at conversation startup to provide soft context for the opening message.

This decision reduced computational load, simplified synchronization, and avoided unnecessary competition for resources with STT, TTS, and LLM inference.

6.2 Model and Inference Context

The project uses a YOLO-based facial emotion detection model. The cited work summaries identify a model checkpoint such as `models/yolov11s_v1.pt` and class metadata provided through a configuration file such as `models/data.yaml`. Earlier experimental work included model evaluation, improvement plans, and comparison of class performance, but the final integrated product uses the emotion model as a lightweight inference module rather than as a continuously running analytical system.

6.3 Bootstrap Detection Procedure

The final camera bootstrap flow is:

1. frontend enters emotion detection mode,
2. camera preview is activated,
3. frontend samples one frame per second over roughly five seconds,

4. each sampled frame is sent to the backend over WebSocket,
5. backend runs inference on each valid frame,
6. predictions are aggregated with a vote over the collected frames,
7. final emotion, confidence, source, and frame statistics are returned,
8. runtime stores the result for opening prompt generation,
9. emotion phase stops,
10. normal conversation begins.

6.4 Why Short Detection Was Better Than Continuous Detection

The project explicitly rejected continuous emotion detection during the full session for the current system scope. The main reasons were:

- unnecessary resource usage,
- increased risk of competition with audio and model services,
- avoidable race conditions between camera and voice pipelines,
- limited marginal value for the demo objective.

A short initial detection window yields enough contextual value for the opening message without destabilizing the rest of the system.

6.5 Multi-Frame Voting

Multi-frame voting is used to improve stability. Instead of trusting a single frame, the backend aggregates predictions across several frames and selects the dominant result. This reduces the influence of transient facial variations, brief camera noise, or low-confidence outliers.

6.6 Output Usage

The final emotion result is used in two places:

1. frontend display, where the user can see the inferred emotion, confidence, and source;
2. runtime prompt context, where the LLM uses the result to generate a more natural opening message.

Chapter 7

Realtime Audio Pipeline

7.1 Transition from File Upload to PCM Streaming

One of the most important architectural transitions in MoodBridge was the move from file- or blob-oriented audio handling to realtime PCM streaming. Earlier approaches based on `MediaRecorder` and uploaded compressed chunks were not appropriate for reliable backend VAD. The final design uses explicit PCM frame transport over WebSocket.

7.2 Audio Format

The PCM streaming path was designed around a backend-friendly format:

- sample rate: 16 kHz,
- channels: mono,
- sample format: 16-bit signed integer,
- frame duration: typically 20 ms,
- base64 encoding for frame payload transport.

7.3 Why PCM Streaming Was Necessary

PCM streaming provides several engineering advantages:

- direct compatibility with VAD,
- predictable frame size and timing,
- simpler utterance segmentation,
- lower ambiguity than browser-compressed formats,
- better fit for a realtime conversation architecture.

7.4 Microphone Lifecycle

The frontend does not record continuously for arbitrary reasons. In the final orchestration model:

- PCM streaming is started when the backend reaches the listening-ready state,
- PCM streaming is stopped or suppressed during opening, thinking, or speaking,
- the backend remains the authority on whether the session should listen.

This prevents a major class of self-listening and timing errors.

Chapter 8

Backend Voice Activity Detection and Utterance Segmentation

8.1 VAD Strategy

The backend uses voice activity detection to decide when user speech starts and ends. This is critical for natural interaction because it allows the system to behave conversationally without requiring manual press-to-talk semantics.

8.2 Core Segmentation Logic

The audio path is conceptually:

```
PCM stream -> decode frame -> VAD -> utterance buffer -> silence threshold
-> speech end -> save WAV -> STT -> cleaned transcript -> runtime
```

8.3 Important Heuristics

During stabilization, several safeguards were added or emphasized:

- **Warmup frames:** ignore or down-weight the earliest frames after stream start.
- **Minimum speech frames:** avoid committing fragments that are too short.
- **Minimum utterance frames:** require a minimum utterance duration before STT.
- **Silence thresholds:** determine when speech has ended.
- **Post-utterance cooldown:** avoid immediate re-triggering after a committed utterance.
- **Volume threshold support:** reduce activation from very low-level noise.

8.4 Why Backend VAD Was the Right Choice

Earlier notes explicitly compare frontend and backend approaches and conclude that backend VAD is more suitable because:

- the backend already owns the runtime,
- VAD decisions should be aligned with state transitions,
- it is easier to tune thresholds and logging centrally,
- it supports future barge-in and advanced policies more cleanly.

8.5 Why Barge-In Was Not Supported in the 1st Version

Although barge-in is an attractive feature for natural voice interaction, it was intentionally excluded from the first complete version of MoodBridge. In this context, *barge-in* refers to allowing the user to interrupt the assistant while the system is still speaking.

This decision was mainly driven by engineering stability rather than by feature limitation. Earlier implementation attempts showed that interruption handling introduces significantly more complexity into the realtime pipeline, especially when the system already depends on synchronized interaction between WebSocket messaging, PCM streaming, backend VAD, STT, LLM response generation, and streaming TTS playback.

Several technical reasons explain why barge-in was deferred.

- the system first needed a stable turn-taking loop before supporting overlapping speech,
- interruption handling requires precise cancellation of TTS playback at the correct time,
- the runtime must decide whether the detected speech is a real user interruption or only residual audio, echo, or device noise,
- barge-in creates race conditions between listening, speaking, and thinking states,
- incorrect interruption behavior can easily damage user experience more than simply requiring turn-based interaction.

In practice, the project encountered related problems even without full barge-in support. One major issue was false-positive speech capture immediately after the system returned to listening mode, especially when PCM streaming restarted shortly after TTS completion. This indicated that the system still needed stronger stabilization around

warmup windows, silence handling, microphone readiness, and post-TTS transition timing before interruption could be implemented safely.

Therefore, the first version adopted a stricter turn-based design:

1. the assistant speaks,
2. TTS finishes completely,
3. the runtime returns to `ARMED_LISTENING`,
4. only then does the frontend resume PCM streaming for user speech.

This design significantly reduces ambiguity in the interaction loop and makes the system easier to debug, evaluate, and stabilize. It also aligns with the broader engineering principle used throughout MoodBridge: first establish a reliable realtime conversation pipeline, then add more advanced interaction behaviors once the core system is stable.

Barge-in remains an important future direction. However, for it to work well, the system will likely require stronger echo suppression, cleaner stream restart timing, more reliable speech validation, and explicit cancellation logic across both TTS and response generation layers.

Chapter 9

Speech-to-Text Integration

9.1 Role of STT

Once the backend commits an utterance, the buffered audio is written to temporary WAV form and transcribed using a speech recognition service, with Whisper referenced in the design summaries.

9.2 Technologies Used in Speech Processing

The speech-to-text functionality in MoodBridge is implemented using a modular pipeline that combines realtime audio streaming, backend-controlled voice activity detection (VAD), and neural speech recognition. Each component was selected to ensure predictable latency, maintainability, and compatibility with the overall conversation runtime.

- **PCM Audio Streaming** Raw microphone audio is captured in the browser and streamed to the backend as base64-encoded PCM frames. This approach provides deterministic timing control and avoids the buffering limitations associated with higher-level browser recording abstractions.
- **Backend Voice Activity Detection (VAD)** Speech segmentation is handled centrally in the backend runtime. The VAD logic monitors incoming audio frames and determines when a user utterance begins and ends based on configurable silence thresholds and speech validation rules. This design ensures that speech boundaries remain synchronized with conversation state transitions.
- **Speech Recognition Model** Once an utterance is finalized, the buffered audio is passed to a speech recognition engine to generate the transcript. The system is designed to support neural ASR models such as **Whisper** or equivalent APIs, allowing flexible deployment depending on performance and resource constraints.

- **Temporary Audio Buffering** Audio frames belonging to a single utterance are accumulated in memory and written to a temporary waveform file before transcription. This intermediate representation simplifies debugging and ensures compatibility with standard speech recognition interfaces.
- **WebSocket-Based Transport Layer** Realtime communication between frontend and backend is implemented using persistent WebSocket connections. This transport mechanism enables continuous audio streaming, low-latency event delivery, and bidirectional synchronization between listening, processing, and response generation phases.

Together, these technologies form a deterministic and extensible speech processing pipeline capable of supporting realtime conversational interaction while maintaining clear separation between audio capture, speech detection, transcription, and conversation logic.

9.3 Transcript Filtering

An important late-stage improvement was the addition of transcript quality filtering before sending user text to the LLM. This was necessary because false triggers or very short noises could otherwise create tiny transcripts that still activated full response generation.

The final filtering logic exists defensively in more than one layer:

- transport or WebSocket layer filtering,
- runtime layer filtering.

This duplication is intentional because it reduces the chance that meaningless text will cause an unnecessary LLM turn.

9.4 Practical Importance

This filtering turned out to be essential in reducing unwanted autonomous loops in which the assistant kept responding to accidental mic captures.

Chapter 10

LLM Integration and Prompting

10.1 Conversation Service Role

The conversation service takes structured context and produces response text. Inputs include:

- detected emotion or latest emotional context,
- user transcript,
- selected conversation language,
- system prompt constraints for style and safety.

10.2 Prompt Philosophy

Across the project documents, prompt design emphasizes the following qualities:

- short responses,
- natural tone,
- empathetic wording,
- no hard emotional certainty,
- no medical claims,
- a conversational rather than overly formal style.

10.3 Opening Message Generation

Opening generation is a special case. The runtime uses the latest detected emotion to generate a first message that acknowledges the user's likely affective state in a gentle, non-diagnostic way. This is one of the defining features of MoodBridge.

10.4 Observed LLM Failure Mode

During integration, an important problem was identified: local LLM requests for the opening message could time out, especially under conditions such as model cold start or backend slowness. This manifested as read timeouts and could appear to users as if the entire session had failed. The lesson here is that AI orchestration errors can masquerade as audio or WebSocket issues if observability is weak.

10.5 Technologies Used and LLM Model Selection

The LLM layer in MoodBridge is implemented through an OpenAI-compatible interface, allowing the conversation runtime to remain model-agnostic while still supporting local deployment. In the current system, the primary model used is a locally hosted open-source large language model, configured through environment variables such as `LLM_MODEL` and `LLM_BASE_URL`. This design allows the runtime to switch between mock mode, compatible API mode, and different backend models without changing the higher-level conversation logic.

The model is used for two closely related tasks: generating the opening message from the detected emotion signal and generating normal conversational responses from user transcripts, emotional context, language preference, and recent conversation history. In both cases, the interaction is structured through prompt templates that enforce short, natural, and non-diagnostic responses. This makes the LLM a reasoning and dialogue generation component rather than a standalone application layer.

10.5.1 Model Selection

In the current system, the primary language model used is **Llama 3**. This model was selected not because it is the largest available model, but because it provides a practical balance between **response quality**, **inference latency**, and **local deployability**.

Compared to alternative models, **Llama 3** is particularly suitable for the goals of MoodBridge for several reasons:

- it produces natural and stable conversational responses,
- it follows prompt instructions reliably, especially for short and empathetic replies,
- it can be deployed locally through OpenAI-compatible interfaces,
- it supports controlled experimentation without relying on external cloud services.

Therefore, **Llama 3** was chosen as a practical model for the current system: sufficiently capable for realtime conversational interaction while remaining flexible for local integration and iterative development.

10.6 Why an Open-Source Model Was Used Instead of a Closed API Model

An open-source model was chosen in the first version of MoodBridge mainly for practical engineering reasons. First, local deployment provides greater control over latency, runtime behavior, and system integration, which is especially important in a realtime multimodal pipeline. Second, it reduces dependency on external cloud services and avoids recurring API costs during development, testing, and repeated debugging cycles. Third, using a self-hosted model makes it easier to inspect failures, tune prompts, and iterate on the conversation pipeline without introducing additional uncertainty from third-party rate limits, policy changes, or network instability.

This choice also aligns with the long-term direction of MoodBridge. Since the project is intended as a foundation for future embedded, robotics, or edge-based interaction systems, relying on an open-source model is more compatible with future offline or semi-offline deployment scenarios. Although closed API models may offer stronger raw performance in some cases, the open-source approach was considered more suitable for experimentation, architectural control, and long-term system ownership.

Chapter 11

TTS Integration

11.1 Role of TTS

The TTS service renders generated text as speech and is tightly coupled to state transitions. While the assistant is speaking, the system should not treat microphone input as valid user speech.

11.2 Streaming-Oriented Design

Project documents describe a streaming TTS philosophy in which text output can be rendered incrementally rather than waiting for the full response. Even when the exact implementation details evolved, the guiding objective remained low perceived latency.

11.3 Listening Suppression During Speaking

A key rule in the final design is simple but critical:

> When TTS is active, listening should be disabled or effectively gated.

This rule prevents self-listening, echo feedback, and repeated turns triggered by the assistant's own output.

11.4 Technologies Used in Text-to-Speech

The text-to-speech (TTS) subsystem in MoodBridge is implemented as a realtime streaming service integrated directly into the conversation runtime. The design prioritizes low perceived latency, predictable state transitions, and stable interaction behavior in a continuous voice conversation environment.

- **Neural Text-to-Speech Engine** Speech synthesis is performed using a neural TTS engine capable of generating natural-sounding speech from text input. The system supports configurable voices and language settings, allowing flexible adaptation to different conversational scenarios. The TTS service operates as a runtime component rather than a standalone application, enabling direct coordination with the conversation state machine.

- **Streaming Audio Generation** Instead of waiting for the full response to be synthesized, the system produces audio incrementally as text becomes available. This streaming-oriented design reduces perceived latency and allows the assistant to begin speaking earlier, improving conversational responsiveness.

- **WebSocket-Based Audio Delivery** Generated audio data is transmitted to the frontend through persistent WebSocket connections. This mechanism enables continuous playback without requiring file downloads or blocking network operations. It also ensures that audio delivery remains synchronized with conversation state updates.

- **State-Coupled Playback Control** The TTS subsystem is tightly integrated with the conversation state machine. When the assistant enters the speaking state, listening is automatically suppressed to prevent self-triggered speech detection. Once playback completes, the runtime transitions back to the listening state and re-enables microphone input.

- **Audio Playback Management** The frontend audio layer handles buffering and playback of streamed audio frames. Playback completion events are propagated back to the backend runtime, ensuring that state transitions occur only after speech output has fully finished. This coordination is essential for maintaining deterministic turn-taking behavior.

Together, these technologies form a synchronized speech output pipeline that supports realtime interaction while preserving system stability and predictable conversation flow.

Chapter 12

Conversation State Machine

12.1 Importance of the State Machine

The conversation state machine is the backbone of MoodBridge. It ensures that frontend and backend have a shared interpretation of what the system is currently doing.

12.2 Core States

Across the project materials, the state names evolved slightly, but the final practical runtime includes the following core states:

- `STOPPED`: the session has been fully terminated and runtime activity is no longer active,
- `READY`: the session has been initialized successfully and the system is prepared to begin conversation,
- `OPENING`: the system is generating or initiating the first assistant message,
- `THINKING`: the assistant is processing user input or preparing a response,
- `SPEAKING`: the assistant is actively rendering speech output,
- `ARMED_LISTENING`: the system is ready to listen, but has not yet confirmed valid user speech,
- `CAPTURING_USER`: the system has detected user speech and is actively collecting the current utterance,
- `PAUSED`: the conversation session remains alive, but active turn-taking is temporarily suspended,
- `ERROR`: the system has encountered an operational failure state requiring graceful handling or recovery.

12.3 Representative Transition Flow

A representative final flow is:

```
Session started
    -> emotion bootstrap
    -> start_conversation
    -> OPENING
    -> SPEAKING
    -> ARMED_LISTENING
    -> CAPTURING_USER (when speech begins)
    -> THINKING (during response generation)
    -> SPEAKING
    -> ARMED_LISTENING
```

12.4 Why `ARMED_LISTENING` Matters

A key debugging insight from later system stabilization sessions was that the runtime often entered the intended listening state successfully but did not remain there long enough to stabilize microphone input. The PCM stream typically restarted immediately after speech output, and early audio frames sometimes contained residual noise, device artifacts, or echo leakage.

As a result, the voice activity detection (VAD) logic could falsely interpret these initial signals as valid speech, causing the system to transition prematurely into the user capture state. This behavior revealed that the problem was not a missing state transition but insufficient signal stabilization after entering the listening state.

The `ARMED_LISTENING` state therefore serves as a readiness phase rather than an active capture phase. Its purpose is to provide a controlled buffer period in which microphone input can stabilize before speech detection begins. This intermediate state significantly reduces false triggers and improves the reliability of turn-taking behavior in realtime conversation systems.

12.5 Design Rationale for the Conversation State Machine

The conversation state machine in MoodBridge was intentionally designed with a moderate number of states to balance clarity, stability, and runtime simplicity. Rather than minimizing the number of states or expanding the system into a highly granular state hierarchy, the final design focuses on representing only the operational phases that materially affect realtime interaction behavior.

Each state corresponds to a distinct physical or computational condition in the system, such as speech playback, user speech capture, or response generation. This mapping ensures that transitions are driven by observable events rather than abstract internal logic, making the system easier to reason about and debug.

The chosen set of states is considered optimal for the current architecture for three primary reasons.

- **Fewer states would reduce behavioral clarity.**

  Combining phases such as listening readiness and active speech capture into a single state would introduce ambiguity in audio handling. For example, treating all listening behavior as a single phase would make it difficult to distinguish between microphone stabilization and confirmed speech detection, increasing the risk of false triggers immediately after playback.

- **More states would increase coordination complexity.**

  Introducing additional micro-states, such as separate states for audio buffering, device warmup, or intermediate inference steps, would require more transition rules and synchronization logic without providing proportional reliability gains. In realtime systems, excessive state granularity often leads to harder debugging and increased risk of inconsistent transitions.

- **The selected states align directly with the natural conversation loop.**

  The final structure reflects the real-world sequence of interaction:

  > system prepares → system speaks → system listens → user speaks → system responds

  By mapping each major interaction phase to a single state, the runtime maintains predictable turn-taking behavior and consistent timing across repeated sessions.

In addition to the core conversational states, the system includes control states such as `STOPPED`, `PAUSED`, and `ERROR`. These states are not part of the normal interaction loop but provide explicit handling for lifecycle termination, temporary suspension, and fault recovery. Their presence improves system robustness and prevents undefined behavior during exceptional conditions.

Overall, the state machine prioritizes operational stability and smooth user interaction over theoretical minimalism. The selected number of states is therefore not arbitrary but the result of iterative refinement during debugging and runtime stabilization. The final configuration represents a practical equilibrium between expressiveness, maintainability, and realtime reliability.

12.6 State Design Trade-Offs

Designing the conversation state machine required balancing competing priorities between **simplicity**, **responsiveness**, **reliability**, and **maintainability**. The final state configuration represents a set of practical **engineering trade-offs** rather than a theoretically minimal or maximally expressive design.

One key trade-off involved the **number of states**. A smaller state set would simplify the diagram and reduce the number of transitions, but it would also reduce the system's ability to distinguish between critical operational phases. For example, merging listening readiness and active speech capture into a single state would make it harder to stabilize microphone input before detecting speech, increasing the likelihood of false triggers.

Conversely, introducing additional micro-states for intermediate steps such as device initialization, audio buffering, or inference preparation would increase the complexity of transition logic without significantly improving runtime behavior. In realtime systems, excessive state granularity can make debugging more difficult and increase the risk of inconsistent transitions.

Another important trade-off involved responsiveness versus stability. Allowing immediate listening after speech output would reduce perceived latency, but it would also increase the probability of detecting residual audio or device noise as valid user speech. The introduction of the `ARMED_LISTENING` state reflects a deliberate decision to prioritize stable interaction over minimal delay.

The design also considers extensibility. While the current system does not include advanced features such as barge-in interruption or multi-user coordination, the existing state structure provides clear extension points for future capabilities. Additional states can be introduced later without restructuring the entire runtime.

Ultimately, the selected state machine reflects a balance between operational clarity and implementation simplicity. The design favors predictable behavior, ease of debugging, and smooth user interaction over theoretical optimality or maximum flexibility.

Chapter 13

WebSocket Protocol Design

13.1 Event-Driven Messaging Model

MoodBridge uses a JSON event model over WebSocket. Messages conceptually follow the shape:

```
{
  "type": "event_name",
  "payload": { ... }
}
```

13.2 Session and Conversation Control Events

Important frontend-to-backend events include:

- `start_session` Initializes a new runtime session and prepares backend resources such as the conversation context, language configuration, and WebSocket state tracking.
- `stop_session` Terminates the current session and releases runtime resources. The system transitions into a stopped state and stops processing incoming audio or conversation events.
- `start_conversation` Signals the backend to begin the conversation loop. This typically triggers the opening message generation phase and prepares the system for interactive turn-taking.
- `pause_conversation` Temporarily suspends the conversation without destroying the session. This event is used to halt interaction while preserving conversation state and session context.

- `set_language` Updates the active language configuration for the session. This affects speech recognition behavior, text generation prompts, and text-to-speech voice selection.

13.3 Audio Streaming Events

Important audio events include:

- `audio_pcm_start` Indicates the beginning of an audio streaming segment. This event informs the backend that incoming audio frames belong to a new user utterance and should be buffered for speech detection.
- `audio_pcm_frame` Carries a chunk of raw PCM audio data from the frontend microphone to the backend runtime. These frames are processed sequentially by the voice activity detection (VAD) logic to determine speech boundaries.
- `audio_pcm_end` Marks the end of the current audio segment. This signal allows the backend to finalize buffering, perform transcription, and transition the conversation state toward response generation.

13.4 Emotion Bootstrap Events

The emotion integration introduced several additional event types:

- `emotion_detection_start` Signals the beginning of the emotion detection phase. The backend prepares the emotion processing pipeline and begins accepting video frames.
- `video_frame` Transfers a single camera frame from the frontend to the backend for emotion analysis. These frames are processed by the computer vision model to estimate the user's emotional state.
- `emotion_detection_end` Indicates that the frontend has stopped sending video frames. The backend completes any pending analysis and prepares the final emotion result.
- `emotion_detection_ready` Notifies the frontend that the backend emotion pipeline is initialized and ready to receive video input.
- `emotion_detection_result` Returns the detected emotional state to the frontend. This result is typically used to guide the generation of the opening conversational message.
- `emotion_update` Provides an updated emotional estimate during ongoing interaction. This event supports dynamic emotional awareness in future system extensions.

13.5 Backend-to-Frontend Response Events

In addition to control and streaming events initiated by the frontend, the backend emits structured response events to communicate runtime progress, system state, and generated outputs. These events allow the frontend to synchronize its user interface and interaction logic with the backend conversation state machine.

- `state_update` Notifies the frontend that the conversation state has changed. This event is the primary synchronization mechanism between backend runtime transitions and frontend behavior, such as enabling or disabling microphone streaming.
- `user_transcript` Delivers the transcribed text generated from captured user speech. The frontend typically displays this text in the conversation history.
- `assistant_response` Sends the generated assistant message to the frontend. This event represents the textual output of the language model before or during speech synthesis.
- `assistant_response_done` Indicates that the assistant response generation phase has completed. This signal helps the frontend determine when the system has finished processing a conversational turn.
- `tts_audio_chunk` Transmits a chunk of synthesized speech audio from the backend to the frontend. These chunks are streamed incrementally to reduce perceived latency during speech playback.
- `tts_audio_end` Signals that the current speech output stream has finished. This event is used to trigger the transition from the speaking phase back to listening readiness.

13.6 Connection and Health Monitoring Events

Realtime conversational systems rely on persistent connections that must remain stable over long interaction sessions. MoodBridge therefore includes dedicated events for connection health monitoring and session reliability.

- `ping` A periodic heartbeat message sent by the frontend to verify that the WebSocket connection remains active.
- `pong` A response message sent by the backend to acknowledge a received ping. Failure to receive a pong within a configured timeout window indicates a connection failure.
- `connection_ready` Indicates that the backend WebSocket handler has successfully initialized and is ready to process incoming events.
- `session_initialized` Confirms that backend session resources have been created successfully.

13.7 Error and Exception Handling Events

To support stable operation in realtime environments, the system defines explicit error signaling mechanisms. These events allow failures to be detected, logged, and handled gracefully without causing undefined behavior.

- `error` Indicates that an operational failure has occurred. The payload typically includes an error message and optional diagnostic details.
- `runtime_error` Signals an internal runtime failure, such as an inference error, audio processing exception, or resource initialization failure.
- `validation_error` Indicates that an incoming message failed schema validation or contained invalid parameters.

13.8 Conversation Lifecycle Events

These events describe transitions in the logical lifecycle of a conversation session and help coordinate interaction flow between system components.

- `conversation_started` Indicates that the conversation loop has been successfully initiated.
- `conversation_paused` Signals that conversation activity has been temporarily suspended.
- `conversation_resumed` Indicates that the system has resumed active interaction after a pause.
- `conversation_stopped` Signals that the conversation session has been fully terminated.

13.9 Why Explicit Events Matter

The explicit event model significantly improved debuggability and modularity across the realtime conversation pipeline. Each message type represents a concrete runtime operation, enabling developers to trace system behavior and verify transitions in a deterministic manner. By separating communication into well-defined events, the system reduces hidden coupling between components and simplifies failure diagnosis during long-running sessions. This design ultimately improves reliability and operational stability in a realtime multimodal interaction environment.

Chapter 14

Final End-to-End User Flow

14.1 Setup and Permissions

A typical final session begins as follows:

1. user opens the web application,
2. browser requests camera and microphone permissions,
3. user selects a language,
4. backend session is established over WebSocket.

14.2 Conversation Bootstrap

Then:

1. frontend begins short emotion detection,
2. backend returns the voted emotion result,
3. frontend issues `start_conversation`,
4. runtime generates the opening response using the detected emotion,
5. TTS speaks the opening,
6. backend transitions into listening-ready mode.

14.3 Normal Dialogue Loop

Then the repeating interaction is:

1. frontend streams PCM only when appropriate,

2. backend detects speech start,
3. backend commits utterance after silence,
4. STT transcribes the utterance,
5. transcript is filtered,
6. runtime generates a response,
7. TTS speaks the response,
8. runtime returns to listening-ready mode.

Chapter 15

Key Engineering Challenges and Debugging Outcomes

15.1 Challenge 1: Camera Flickering and Missing Frames

During YOLO integration, a major problem appeared: the webcam repeatedly flickered and the backend received no usable frames. The root cause was not in the backend but in the frontend camera component. An effect was depending on callback references that changed on re-render, causing the camera stream to start and stop repeatedly.

15.1.1 Resolution

The fix was to stabilize the camera lifecycle:

- store the latest callback in a ref,
- make the camera effect depend only on durable enable state,
- avoid inline callback references for video readiness.

This restored stable frame capture and allowed the backend to receive image data consistently.

15.2 Challenge 2: Emotion Detection Triggering at the Wrong Time

After initial integration, the emotion bootstrap phase could be triggered more than once or at incorrect times, including after the opening phase had already completed.

15.2.1 Resolution

The frontend orchestration was hardened using ref-based guards that tracked:

- whether emotion detection was already in flight,
- whether conversation bootstrap was pending,
- whether the conversation startup process was already running.

This ensured that emotion detection occurred only once in the intended location of the session flow.

15.3 Challenge 3: Microphone Selection and Readiness

A particularly difficult frontend issue involved incorrect or unstable microphone selection. Even with permission granted, the browser could initialize capture using the wrong device if `deviceId` was not specified, and device enumeration timing could leave the UI in a partially ready state.

15.3.1 Resolution

The recorder was extended to accept an explicit device identifier, and the UI was updated to support microphone selection. Additional guards and retry logic were introduced to ensure that PCM streaming would not start until the chosen device was actually available.

15.4 Challenge 4: Auto-Stop and Sudden Disconnects

One of the most important debugging outcomes in the project was the identification of WebSocket keepalive timeout as a key source of sudden session death. This had originally seemed like a conversation logic issue, but logs showed that the socket often disconnected first, after which backend attempts to send events caused secondary exceptions.

15.4.1 Resolution

The development server was stabilized using adjusted Uvicorn keepalive settings, and safer backend send logic was introduced. The confirmed development command was:

```
uvicorn app.main:app \
  --reload \
  --ws-ping-interval 60 \
  --ws-ping-timeout 60 \
  --timeout-keep-alive 120
```

This was a major diagnostic milestone because it clarified that transport stability, not just runtime logic, mattered critically.

15.5 Challenge 5: False Speech Capture After Listening

Even after the system reached the armed-listening state correctly, it could immediately re-enter capture mode from silence or low-level noise, especially shortly after TTS completion.

15.5.1 Resolution Status

This issue was mitigated but not entirely eliminated. Improvements included:

- warmup windows,
- cooldown logic,
- minimum utterance thresholds,
- short transcript filtering,
- tighter PCM lifecycle control.

The insight that the system *was* entering the correct state but failing to *hold* it stably narrowed the problem significantly.

15.6 Challenge 6: Local LLM Timeout During Opening

At times, opening generation via a local LLM endpoint timed out, making the system appear to have failed for audio reasons.

15.6.1 Resolution Status

The practical conclusion was that LLM cold start, local load, or a short timeout could produce apparent session collapse. Future work should include longer timeout windows and graceful fallback rather than letting opening failure kill the experience.

15.7 Challenge 7: Missed User Speech After Entering `ARMED_LISTENING`

During integration of the emotion detection pipeline and full conversation flow, an intermittent issue was observed: the system successfully transitioned into the `ARMED_LISTENING` state, the microphone permission had already been granted, yet user speech was not detected immediately after the transition.

This behavior created the impression that the listening system had failed, even though the state machine and voice activity detection logic were functioning correctly. The root cause was not related to audio processing, but to the ordering of hardware initialization and state transitions in the frontend interaction flow.

Specifically, the system entered the listening readiness state before the browser had fully initialized the microphone stream. Although permission had been granted, the audio device required a short initialization period before producing valid PCM frames. As a result, the first user utterance could occur during this gap, leading to missed speech detection.

15.7.1 Resolution

The fix involved adjusting the initialization sequence so that microphone readiness was established before transitioning into the listening state. The frontend now requests microphone permission and initializes the audio stream earlier in the session lifecycle, ensuring that the device is fully prepared when the runtime enters `ARMED_LISTENING`.

- request microphone permission during session setup rather than after state transition,
- wait for the audio stream to become active before enabling listening readiness,
- ensure that hardware readiness aligns with runtime state transitions.

This change eliminated missed initial speech events and significantly improved responsiveness during the first user turn.

Chapter 16

UI and UX Stabilization

16.1 Conversation UI Evolution

The frontend UI evolved from an early debug-oriented interface into a clearer product-style conversation screen designed to support realtime multimodal interaction. The final layout organizes the interface into distinct functional regions that reflect the runtime architecture and state machine behavior.

The production interface includes the following primary sections:

- a central visual orb representing the assistant state,
- a camera preview panel used for realtime emotion detection,
- a conversation stream displaying assistant and user messages,
- a control board for managing backend and conversation sessions,
- system and status panels showing device readiness and runtime state,
- microphone device selection and language switching controls.

This structure aligns the visual layout with the operational phases of the conversation runtime, allowing users and developers to quickly understand the current system condition during live interaction.

16.2 Orb Behavior and State Synchronization

The visual orb component serves as a direct representation of the conversation state machine. Rather than functioning as a purely decorative animation, the orb reflects the assistant's operational phase in realtime. For example, the orb transitions visually between states such as:

- idle or ready state,

- thinking or processing state,
- speaking state,
- listening readiness state.

Earlier prototypes implemented complex animated effects and dynamic rendering behaviors intended to create a more visually expressive interface. However, these effects introduced unnecessary rendering overhead and occasionally contributed to increased CPU usage and event loop latency during intensive interaction cycles.

In a realtime system where backend responsiveness and audio timing are critical, excessive frontend rendering complexity can indirectly affect system stability. Heavy visual updates may delay event processing, increase resource consumption, or contribute to timing inconsistencies during speech generation and streaming.

As a result, the orb implementation was intentionally simplified in the final version. The design now prioritizes predictable rendering behavior and stable performance over visual complexity. This adjustment reduced the likelihood of timeouts or delayed responses during long-running conversation sessions and helped maintain consistent interaction flow between frontend and backend.

16.3 Session Visibility and Control Flow

A key usability improvement in the final UI was the introduction of explicit session visibility and control mechanisms. The interface clearly distinguishes between system readiness, conversation activity, and device availability.

The control board allows operators to manage the runtime lifecycle through separate toggles for:

- backend service activation,
- conversation session start and stop,
- microphone device selection,
- language selection.

This separation reflects the underlying architecture, where backend services, conversation logic, and hardware devices are managed independently. By exposing these controls directly in the UI, the system reduces ambiguity about whether a failure originates from the network, runtime, or device layer.

16.4 System Status and Observability

The final interface includes a dedicated status panel that displays the current state of critical system components. These indicators provide immediate feedback about runtime readiness and help diagnose issues during development and testing.

Typical status indicators include:

- microphone permission status,
- camera permission status,
- server connectivity,
- conversation activity state,
- current conversation state machine value.

This explicit visibility significantly improved observability during integration and debugging. Developers could quickly determine whether the system was waiting for hardware readiness, processing audio, or generating responses.

16.5 Key Practical Improvements

Several UI-level refinements proved meaningful even though they were not part of the core AI logic. These adjustments improved reliability, usability, and system transparency during realtime interaction.

- clearer visualization of runtime state transitions,
- separation of system messages from primary dialogue content,
- consistent labeling of the assistant as `MOODBRIDGE`,
- explicit display of emotion detection results and confidence values,
- improved microphone device selection controls,
- simplified language switching interface,
- reduced visual rendering complexity to improve runtime stability.

Together, these changes transformed the interface from a development prototype into a stable operational frontend capable of supporting continuous realtime conversation sessions.

Chapter 17

Deployment and Operational Notes

17.1 Environment Setup Strategy

Project notes emphasize that the environment should reflect actual working dependencies instead of being reconstructed manually from memory. The recommended practice is to synchronize files such as `environment.yml` and `requirements.txt` from the real environment.

17.2 CLI and Script Workflow

Operational guides describe a simple shell-based workflow:

- create or activate the Conda environment,
- optionally install remaining packages from `requirements.txt`,
- load LLM environment configuration,
- run the main app via helper scripts,
- run standalone demos via a secondary script if needed,
- stop services using a stop script or terminal interruption.

17.3 Practical Development Command

Because package layout matters, backend commands should be run from the correct directory, and the recommended development launch settings include extended WebSocket keepalive parameters, as shown earlier.

Chapter 18

Current System Capabilities

18.1 What the Final System Can Do

In its final integrated configuration, MoodBridge operates as a realtime, event-driven conversational system that combines computer vision, speech processing, and language generation into a coordinated interaction loop. The system is capable of executing a complete multimodal conversation cycle from user presence detection to spoken response delivery.

The current implementation supports the following functional capabilities:

- initialize a browser-based session and establish a persistent WebSocket connection with the backend runtime,
- request and manage hardware permissions for camera and microphone devices during session setup,
- capture webcam frames during a short emotion bootstrap phase,
- perform realtime facial emotion inference using a YOLO-based computer vision model,
- select the most stable detected emotion and expose it as the initial conversational context,
- automatically generate an opening message that reflects the detected emotional state in a non-diagnostic and user-friendly manner,
- maintain a structured conversation lifecycle controlled by a deterministic state machine,
- stream microphone audio from the browser to the backend as time-aligned PCM frames,
- segment user speech using backend-controlled voice activity detection (VAD) based on configurable silence thresholds,

- finalize user utterances after validated silence periods,
- transcribe speech using a neural speech recognition service,
- filter and normalize transcripts before passing them to the language model,
- generate conversational responses using a locally hosted open-source large language model,
- synthesize speech output using a streaming text-to-speech service,
- coordinate listening suppression during speech playback to prevent self-triggered responses,
- maintain synchronization between audio streaming, inference, and state transitions,
- display conversation history and assistant responses in a structured conversation panel,
- visualize the current runtime state using a dynamic orb component,
- display realtime system readiness indicators for camera, microphone, server connectivity, and conversation state,
- support manual control of session lifecycle through explicit frontend toggles,
- recover gracefully from common runtime interruptions such as temporary audio loss, delayed responses, or transient connection issues.

These capabilities collectively enable the system to sustain a continuous, interactive dialogue session without requiring manual intervention between turns.

18.2 Conversation Flow Stability

The final system is capable of maintaining a stable turn-taking loop under typical usage conditions. Each interaction cycle follows a predictable sequence of operational phases:

1. emotion detection bootstrap,
2. system readiness confirmation,
3. assistant message generation,
4. speech playback,
5. listening readiness transition,
6. user speech capture,

7. speech transcription,
8. response generation,
9. speech synthesis,
10. return to listening readiness.

This structured loop allows the runtime to coordinate multiple asynchronous subsystems while preserving deterministic interaction behavior.

The introduction of explicit states such as `ARMED_LISTENING` and `CAPTURING_USER` significantly improved system stability by separating listening readiness from active speech capture.

18.3 Operational Constraints

While the system is capable of sustained realtime interaction, several operational constraints remain inherent to the current prototype-level implementation:

- long conversation sessions may increase memory usage due to accumulated audio buffers and message history,
- response latency may vary depending on local hardware performance and model initialization state,
- the emotion detection model is optimized for demonstration scenarios rather than clinical accuracy,
- the system currently prioritizes English interaction quality, with Vietnamese support still under refinement,
- network instability or device interruptions can temporarily degrade responsiveness.

These constraints reflect typical behavior for realtime multimodal systems operating in a research and development environment.

18.4 System Reliability Features

Several engineering mechanisms were implemented to improve runtime reliability and observability:

- explicit conversation state machine transitions,
- centralized runtime orchestration,

- event-driven communication over WebSocket,
- heartbeat-based connection monitoring,
- controlled listening suppression during speech playback,
- structured error handling and fallback behavior,
- visible system status indicators in the user interface,
- early hardware initialization to prevent missed input events.

These mechanisms reduce the likelihood of silent failures and make system behavior easier to diagnose during live interaction.

18.5 System Maturity Assessment

The most accurate maturity description of the current MoodBridge system is:

> A demo-ready, end-to-end, realtime multimodal conversational AI system with a stable interaction architecture and clearly defined runtime control model, capable of supporting continuous user dialogue under controlled conditions, but still requiring further optimization, scalability testing, and deployment hardening for production-level operation.

In practical terms, the system has successfully demonstrated:

- full integration of vision, speech, and language components,
- stable multi-turn conversation handling,
- deterministic state-driven runtime behavior,
- recoverable operation under common runtime disruptions,
- consistent user interaction without manual system resets.

The remaining work focuses primarily on performance optimization, resource management, and long-duration session reliability rather than fundamental architectural changes.

Chapter 19

Known Limitations

19.1 Behavioral Limitations

Although the system demonstrates stable realtime interaction under typical conditions, several behavioral limitations remain inherent to the current prototype implementation. These limitations primarily relate to timing, resource coordination, and asynchronous system behavior.

- false speech capture may still occur shortly after entering the `ARMED_LISTENING` state if residual audio or environmental noise is interpreted as valid speech,
- local language model initialization latency can delay the opening response during cold start conditions,
- microphone readiness and browser device enumeration remain timing-sensitive and may introduce short initialization delays on resource-constrained systems,
- rapid state transitions under heavy CPU load can temporarily affect responsiveness during long conversation sessions,
- prolonged interaction sessions may gradually increase memory usage due to accumulated conversation history and buffered audio data,
- system responsiveness can vary depending on hardware performance, particularly when running multiple inference services concurrently.

These behavioral limitations are typical for realtime multimodal systems that coordinate multiple asynchronous components within a single runtime.

19.2 Technical Limitations

Several technical constraints arise from architectural and infrastructure choices made during development. These limitations do not prevent system operation but define the

boundaries of current reliability and scalability.

- the system relies on a single-node runtime architecture without distributed processing or load balancing,
- GPU acceleration is not yet fully utilized across all inference components,
- model inference and audio processing share system resources, which can introduce contention under heavy workloads,
- network communication is optimized for local development rather than high-latency or unreliable network environments,
- logging and monitoring mechanisms are designed primarily for debugging rather than production observability,
- automatic restart and watchdog mechanisms are not yet implemented.

These constraints reflect the system's role as an integrated prototype rather than a fully hardened production platform.

19.3 Operational Limitations

Operational limitations describe constraints related to runtime stability, deployment readiness, and real-world usage scenarios.

- continuous multi-hour sessions have not yet been fully stress-tested under sustained production workloads,
- system startup time can vary depending on model loading and device initialization latency,
- hardware dependency remains significant, particularly for audio and video device compatibility,
- system performance may degrade on low-resource machines or shared development environments,
- deployment procedures currently require manual environment setup and configuration.

These operational constraints are common during the transition from research prototype to production-ready system.

19.4 Feature Limitations

Several advanced capabilities remain intentionally out of scope for the current system version or are only partially implemented. These features represent natural next steps in the system's evolution.

- robust barge-in support allowing users to interrupt assistant speech in realtime,
- long-term persistent conversational memory across sessions,
- multi-user session management with authentication and user isolation,
- centralized database storage for conversation logs and analytics,
- advanced emotion tracking over time rather than single-frame inference,
- multilingual optimization beyond the current primary English workflow,
- automated model selection or dynamic performance scaling,
- full production deployment topology including containerization, orchestration, and monitoring infrastructure.

These limitations reflect design prioritization rather than missing functionality. The current system intentionally focuses on stable realtime interaction before expanding into advanced feature domains.

19.5 Design Trade-Off Summary

Many of the limitations listed above are the direct result of deliberate engineering trade-offs made during development. The system prioritizes deterministic behavior, observability, and integration clarity over maximum feature complexity.

Key trade-offs include:

- prioritizing runtime stability over aggressive performance tuning,
- simplifying visual rendering to reduce frontend resource overhead,
- deferring advanced features such as barge-in to maintain predictable state transitions,
- focusing on local deployment to simplify debugging and development,
- emphasizing explicit state control rather than automated adaptation.

These trade-offs allowed the system to reach a stable and demonstrable end-to-end architecture while preserving a clear path for future enhancements.

Chapter 20

Future Work

20.1 Short-Term Priorities

Short-term work focuses on improving reliability, responsiveness, and developer experience without fundamentally changing the system architecture. These tasks primarily address stability and operational robustness in the current runtime.

The most valuable short-term improvements include:

1. fully stabilize microphone readiness and device selection across browsers and operating systems,
2. improve hold stability in the `ARMED_LISTENING` state to reduce false triggers caused by residual audio or environmental noise,
3. implement graceful timeout handling for local language model responses, particularly during cold start conditions,
4. preserve session liveness even when one subsystem temporarily fails (for example, camera, speech recognition, or network connectivity),
5. finalize reproducible startup scripts and environment configuration to simplify system deployment and demonstration setup,
6. standardize runtime logging formats to improve debugging consistency,
7. validate system behavior under repeated start–stop session cycles.

These improvements strengthen operational reliability and reduce friction during live demonstrations and extended interaction sessions.

20.2 Medium-Term Priorities

Medium-term development focuses on improving system usability, observability, and performance optimization while maintaining the current architectural model.

Useful medium-term work includes:

1. refine prompt design strategies to produce more natural and context-aware emotion-based opening messages,
2. provide clearer microphone readiness and device status feedback in the UI, including explicit readiness confirmation before entering listening states,
3. implement structured logging and diagnostic dashboards for runtime monitoring and performance analysis,
4. introduce application-level heartbeat or watchdog mechanisms to detect and recover from stalled components,
5. separate heavy computation tasks from the main event loop using worker threads or asynchronous task queues,
6. improve resource management for audio buffers and message history,
7. enable optional GPU acceleration for emotion detection and speech processing components,
8. optimize conversation state transitions to reduce perceived latency.

These changes aim to improve system responsiveness and maintainability without requiring major architectural redesign.

20.3 Long-Term Directions

Long-term development expands the system beyond its current prototype scope toward scalable, adaptive, and research-oriented capabilities.

Potential long-term directions include:

1. revisiting periodic emotion re-detection during extended conversations to support dynamic emotional awareness,
2. integrating multi-modal fusion techniques combining facial expression, voice tone, and conversational context signals,
3. introducing separate inference queues for emotion detection, speech recognition, and language generation to improve concurrency and scalability,

4. designing a distributed deployment architecture supporting multiple concurrent users,
5. implementing persistent conversational memory and user-specific interaction history,
6. developing a centralized database for conversation logging and analytics,
7. improving multilingual support, particularly for Vietnamese speech recognition and conversational response generation,
8. introducing adaptive system behavior based on long-term user interaction patterns,
9. improving model efficiency through quantization, model pruning, or hardware-specific optimization,
10. conducting deeper research into emotion recognition quality, temporal smoothing, and uncertainty handling.

These directions position the system as a foundation for future research and applied human–AI interaction systems.

20.4 Application-Oriented Expansion

Beyond technical improvements, the MoodBridge architecture can support a range of applied scenarios that extend the system's impact beyond a single prototype.

Potential application-oriented expansions include:

1. emotion-aware conversational agents for mental wellness support,
2. interactive educational assistants capable of adapting to student engagement levels,
3. human–robot interaction systems where robots respond to emotional cues,
4. assistive technologies for communication support in healthcare or therapy environments,
5. social companion systems for long-duration interaction settings.

These applications demonstrate how the system's core architecture can evolve from a demonstration prototype into a practical human-centered interaction platform.

20.5 Strategic Vision

The long-term vision for MoodBridge is to develop a stable and extensible realtime conversational system capable of understanding emotional signals and responding in a natural, context-aware manner.

Rather than focusing solely on individual models, the system emphasizes integration, synchronization, and reliability across multiple sensing and interaction modalities. Future development will prioritize robustness, scalability, and ethical human-centered design while preserving the deterministic runtime behavior established in the current architecture.

Chapter 21

System Technical Stack

The MoodBridge system is implemented as a realtime multimodal conversational platform that integrates web technologies, audio processing frameworks, machine learning models, and asynchronous communication infrastructure. The architecture combines browser-based interaction, backend orchestration, and AI-driven speech and emotion processing into a unified runtime environment.

The technical stack can be categorized into frontend technologies, backend infrastructure, artificial intelligence components, communication mechanisms, and system-level runtime tools.

21.0.1 Frontend Technologies

The frontend interface is implemented as a browser-based realtime interaction environment designed to support continuous conversation and system visibility.

- **React** React is used as the primary user interface framework. It enables a modular component-based architecture for rendering conversation streams, camera preview, system status panels, and session controls. The framework supports efficient state-driven rendering required for realtime UI updates.
- **TypeScript** TypeScript is used across the frontend to define explicit data structures, particularly for WebSocket message contracts and runtime state synchronization. Strong typing improves reliability and maintainability in asynchronous systems.
- **Vite** Vite serves as the frontend development server and build toolchain. It provides fast startup time, efficient module bundling, and rapid iteration during UI development and debugging.
- **Tailwind CSS** Tailwind CSS is used as the styling framework for the user interface. It enables consistent layout design and responsive component styling with minimal runtime overhead.

- **WebSocket API** The browser WebSocket API enables persistent bidirectional communication between the frontend and backend runtime. This mechanism supports realtime message exchange, audio streaming, and state synchronization.
- **Browser MediaDevices API (getUserMedia)** The MediaDevices API is used to request microphone and camera access from the browser. It enables low-latency audio capture and video preview directly within the web environment.
- **Web Audio Processing** Custom audio processing logic is implemented in the browser to capture microphone input, downsample audio signals, convert audio to mono `int16` format, segment audio into time-aligned frames, and encode audio frames for transmission.

21.0.2 Backend Technologies

The backend runtime is responsible for orchestrating conversation flow, managing system state transitions, and coordinating AI inference services.

- **Python** Python is used as the primary backend programming language. It provides a flexible environment for asynchronous processing, AI model integration, and system control.
- **FastAPI** FastAPI is used as the backend web framework to manage WebSocket endpoints, session lifecycle, and asynchronous request handling. Its native support for `asyncio` makes it suitable for realtime streaming applications.
- **Starlette WebSocket Layer** The WebSocket implementation relies on Starlette's asynchronous communication layer, which handles connection management and disconnection events.
- **Pydantic** Pydantic is used for validating WebSocket message schemas and enforcing structured data exchange between frontend and backend components.
- **asyncio** The backend runtime uses Python's asyncio framework to coordinate concurrent operations such as audio streaming, inference execution, and session management.
- **NumPy** NumPy is used for efficient numerical processing of audio buffers and image data.
- **OpenCV (cv2)** OpenCV is used for image decoding and preprocessing of camera frames before emotion inference.
- **PyYAML** PyYAML is used to load configuration files such as model metadata and class labels.
- **requests** The requests library is used to communicate with external or local language model services through HTTP-based APIs.

21.0.3 Artificial Intelligence and Machine Learning Components

The system integrates multiple AI components responsible for perception, language generation, and speech interaction.

- **Ultralytics YOLO** A YOLO-based deep learning model is used for facial emotion detection. The model processes camera frames during the emotion bootstrap phase to infer an initial emotional state.
- **Whisper Speech Recognition Model** The Whisper model is used for speech-to-text conversion. It transcribes user speech after voice activity detection confirms the end of an utterance.
- **Voice Activity Detection (webrtcvad)** The webrtcvad library is used to detect speech boundaries based on audio energy and silence thresholds. It determines when user speech begins and ends.
- **Text-to-Speech Engine (edge-tts)** The edge-tts neural speech synthesis engine is used to convert generated text into spoken audio responses. Streaming audio generation reduces perceived latency.
- **Large Language Model (LLM)** A locally hosted open-source large language model is used to generate conversational responses. The model is accessed through an OpenAI-compatible API interface.
- **Llama 3 Model** The deployed language model is based on the Llama 3 architecture, which provides high-quality conversational capabilities while supporting local inference.
- **Ollama Runtime** Ollama is used as the local model serving environment. It manages model loading, inference execution, and API exposure for language generation services.

21.0.4 Audio and Communication Technologies

Realtime audio processing and communication are central to the system's operation.

- **PCM Audio Streaming** Raw microphone audio is captured and streamed as PCM frames from the browser to the backend. This low-level streaming approach provides deterministic timing control and enables precise synchronization with the conversation state machine.
- **Base64 Audio Encoding** Audio frames are encoded as base64 strings for transmission over WebSocket connections.
- **WebSocket Protocol** Persistent WebSocket communication enables realtime event-driven interaction between frontend and backend components.

- **Ping / Pong Heartbeat Mechanism** A periodic heartbeat mechanism is implemented to monitor connection health and maintain session liveness during long-running conversations.

21.0.5 System and Runtime Environment

The system operates within a controlled runtime environment designed to support reproducible development and deployment.

- **Ubuntu Linux** The system is developed and tested on Ubuntu Linux, which provides stable support for Python environments, audio devices, and machine learning frameworks.
- **Conda Environment Management** Conda is used to manage Python dependencies and isolate runtime environments. This ensures consistent library versions across development and testing systems.
- **Shell Scripts** Shell scripts are used to automate system startup and shutdown procedures, including launching backend services and initializing runtime components.
- **Node.js** Node.js is used as the runtime environment for frontend development tools.
- **npm (Node Package Manager)** npm is used to install and manage frontend dependencies such as React, TypeScript, and Vite.

21.0.6 Summary

The MoodBridge technical stack combines modern web technologies, asynchronous backend infrastructure, and multiple AI inference components into a unified realtime interaction system. The integration of browser-based media capture, event-driven communication, and locally hosted AI models enables the system to operate as a fully self-contained conversational platform without relying on external cloud services.

Chapter 22

Conclusion

MoodBridge successfully evolved from a collection of experimental modules into a coherent realtime conversational AI system that integrates computer vision, audio streaming, speech recognition, language generation, and speech synthesis into a single browser-accessible workflow. The system demonstrates how multiple AI components can be orchestrated through a structured runtime model to produce a stable and responsive user interaction experience.

One of the most significant architectural milestones in the project was the transition from loosely coupled scripts to a backend-owned runtime governed by an explicit conversation state machine and an event-driven communication model. Adopting a WebSocket-first transport layer enabled deterministic control over audio streaming, inference timing, and session lifecycle management. This shift introduced a clear separation of responsibilities between frontend and backend components, improved observability, and allowed systematic debugging of issues related to camera initialization, microphone readiness, speech segmentation, and realtime conversation flow.

Another key achievement was the stabilization of the user interaction loop. Through iterative refinement of the state machine, the system established a predictable turn-taking mechanism that synchronizes listening, thinking, and speaking phases. The introduction of intermediate states such as `ARMED_LISTENING` and `CAPTURING_USER` proved particularly important for reducing false triggers and improving reliability in realtime audio environments. These design decisions illustrate how careful state management can significantly enhance the robustness of multimodal systems.

From a user interaction perspective, the final product demonstrates a compelling multimodal conversation pattern. The system begins by observing a brief facial expression signal during an emotion bootstrap phase, uses that signal as a soft context for generating an opening message, and then continues the dialogue through realtime spoken interaction. The integration of visual, auditory, and language components into a continuous interaction loop represents a meaningful step toward more natural human–AI communication.

Equally important were the engineering trade-offs made throughout development. Several complex features were intentionally simplified or deferred in order to prioritize runtime stability and predictable behavior. For example, the visual orb interface was redesigned to reduce rendering overhead, barge-in capability was postponed to maintain deterministic state transitions, and local model deployment was chosen to improve control over latency and debugging conditions. These decisions reflect a deliberate focus on reliability and system coherence rather than feature complexity.

At its current stage, MoodBridge is best understood as a polished prototype with a production-oriented architectural foundation. The system is capable of demonstrating the full interaction pipeline end-to-end, sustaining multi-turn conversations, and recovering from common runtime interruptions. While further optimization and deployment hardening are required for large-scale production use, the core architecture has proven stable, extensible, and suitable for continued development.

Beyond its technical implementation, the project provides valuable engineering lessons about building realtime media systems. It highlights the importance of explicit state control, hardware readiness synchronization, structured event communication, and incremental system stabilization. These lessons extend beyond the specific application domain and contribute to a broader understanding of how complex AI services can be integrated into reliable, user-facing interaction systems.

In summary, MoodBridge demonstrates that meaningful progress in conversational AI does not rely solely on individual models, but on the careful coordination of sensing, processing, and interaction components. The system establishes a solid foundation for future exploration in emotion-aware communication, human–robot interaction, and multimodal AI systems designed to support natural and responsive human experiences.

Appendix A

MoodBridge System Diagram

The following directory tree illustrates the overall organization of the MoodBridge system. The structure reflects the separation between frontend, backend runtime components, service modules, and supporting resources.

```
moodbridge/
|-- backend/
|   |-- test_ws_client.py
|   `-- app/
|       |-- main.py
|       |-- runtime/
|       |   |-- voice_input_controller.py
|       |   |-- conversation_runtime.py
|       |   `-- runtime_services.py
|       |-- state/
|       |   `-- conversation_state.py
|       |-- schemas/
|       |   `-- ws_messages.py
|       `-- ws/
|           `-- conversation_ws.py
|-- services/
|   |-- conversation_service.py
|   |-- emotion_service.py
|   |-- audio_recorder.py
|   |-- sounddevice_recorder.py
|   |-- speech_service.py
|   `-- tts_service.py
|-- frontend/
|   `-- src/
|       |-- components/
```

```
|       |   |-- camera/
|       |   |   `-- CameraPreviewCard.tsx
|       |   |-- conversation/
|       |   |   `-- ConversationStream.tsx
|       |   |-- orb/
|       |   |   `-- VoiceOrb.tsx
|       |   |-- setup/
|       |   |   |-- LanguageSwitch.tsx
|       |   |   |-- StatusPanel.tsx
|       |   |   |-- SystemPanel.tsx
|       |   |   `-- VoiceSessionPanel.tsx
|       |   `-- ui/
|       |       `-- ToggleSwitch.tsx
|       |-- lib/
|       |   `-- pcmRecorder.ts
|       |-- pages/
|       |   `-- ConversationPage.tsx
|       `-- types/
|           `-- ws.ts
|-- demos/
|   |-- demo_conversation.py
|   |-- demo_speech.py
|   `-- demo_webcam.py
|-- models/
|   |-- yolov11s_v1.pt
|   `-- data.yaml
|-- scripts/
|   |-- run_main.sh
|   |-- run_subs.sh
|   `-- stop_main.sh
|-- tools/
|   |-- test_audio_devices.py
|   |-- test_edge_tts.py
|   `-- test_llm_api.py
|-- environment.yml
|-- requirements.txt
`-- README.md
```

This directory structure reflects the modular architecture of the system, separating runtime orchestration, service components, frontend interaction, and supporting utilities into clearly defined functional layers.

A.1 System Architecture Diagram

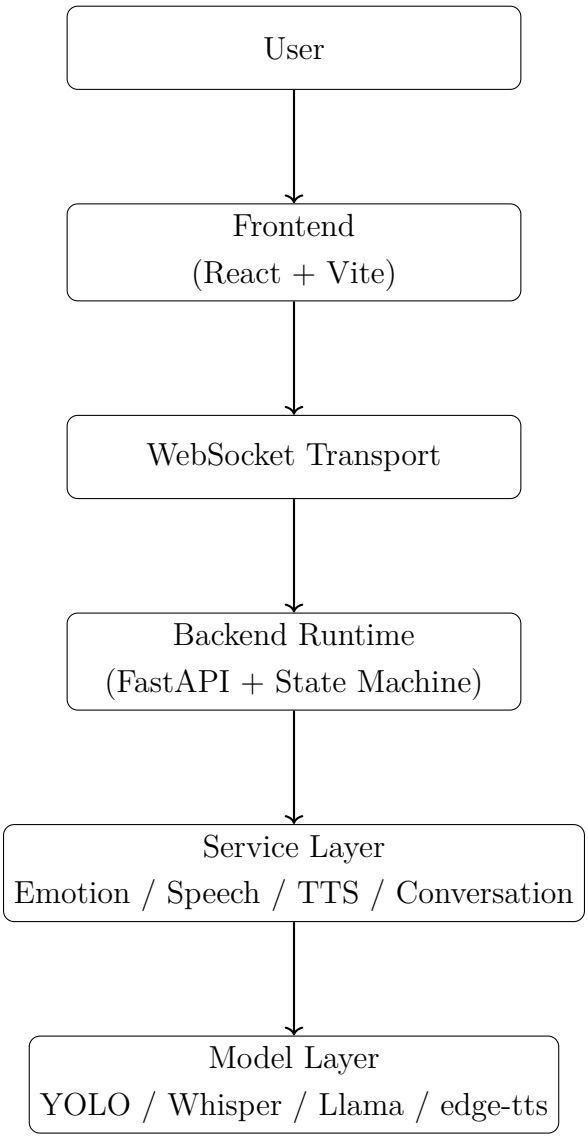


Figure A.1: High-level system architecture of the MoodBridge realtime conversational AI system.

A.2 Conversation State Machine Diagram

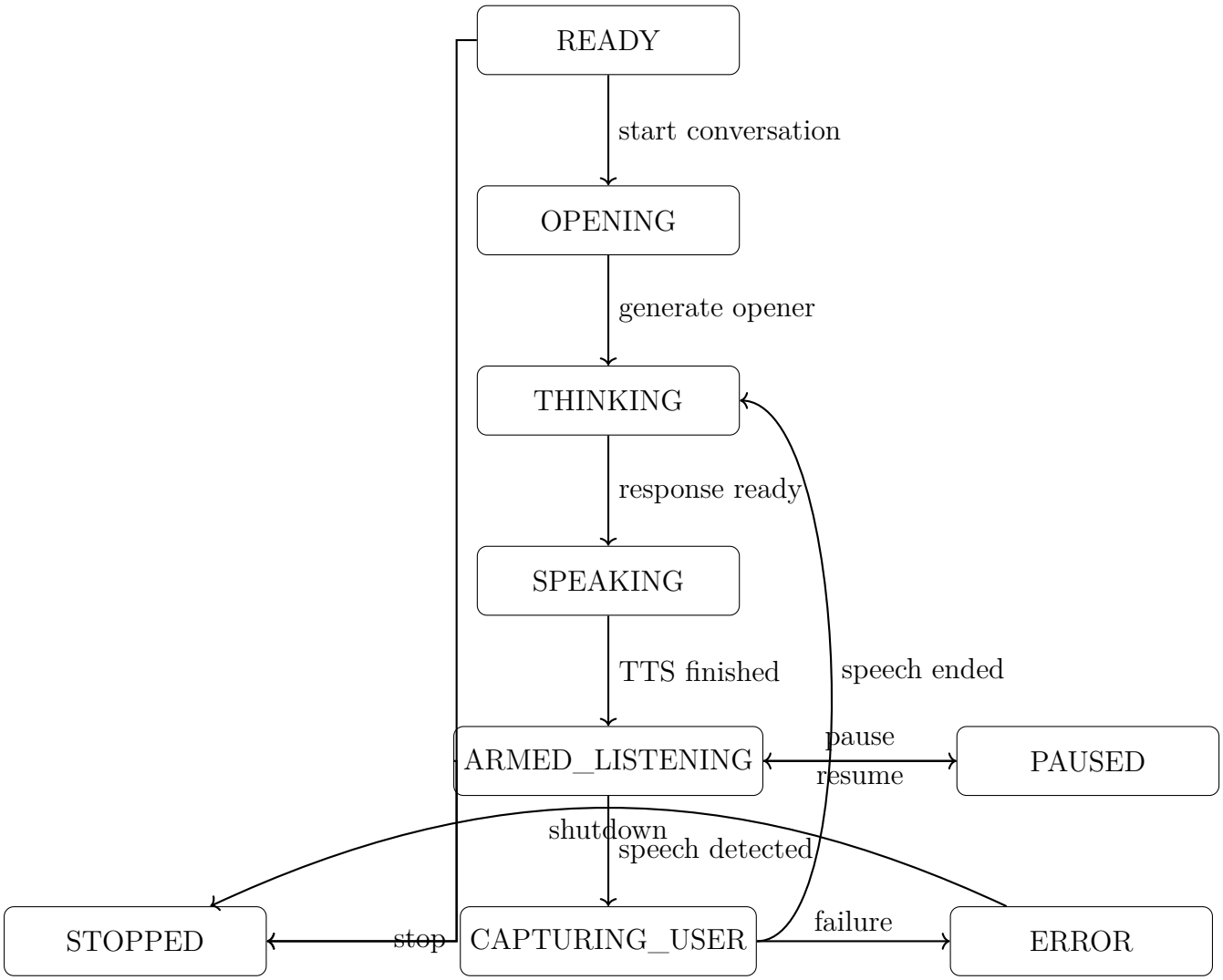


Figure A.2: Conversation state machine governing realtime interaction flow in MoodBridge.

A.3 Conversation Interaction Sequence

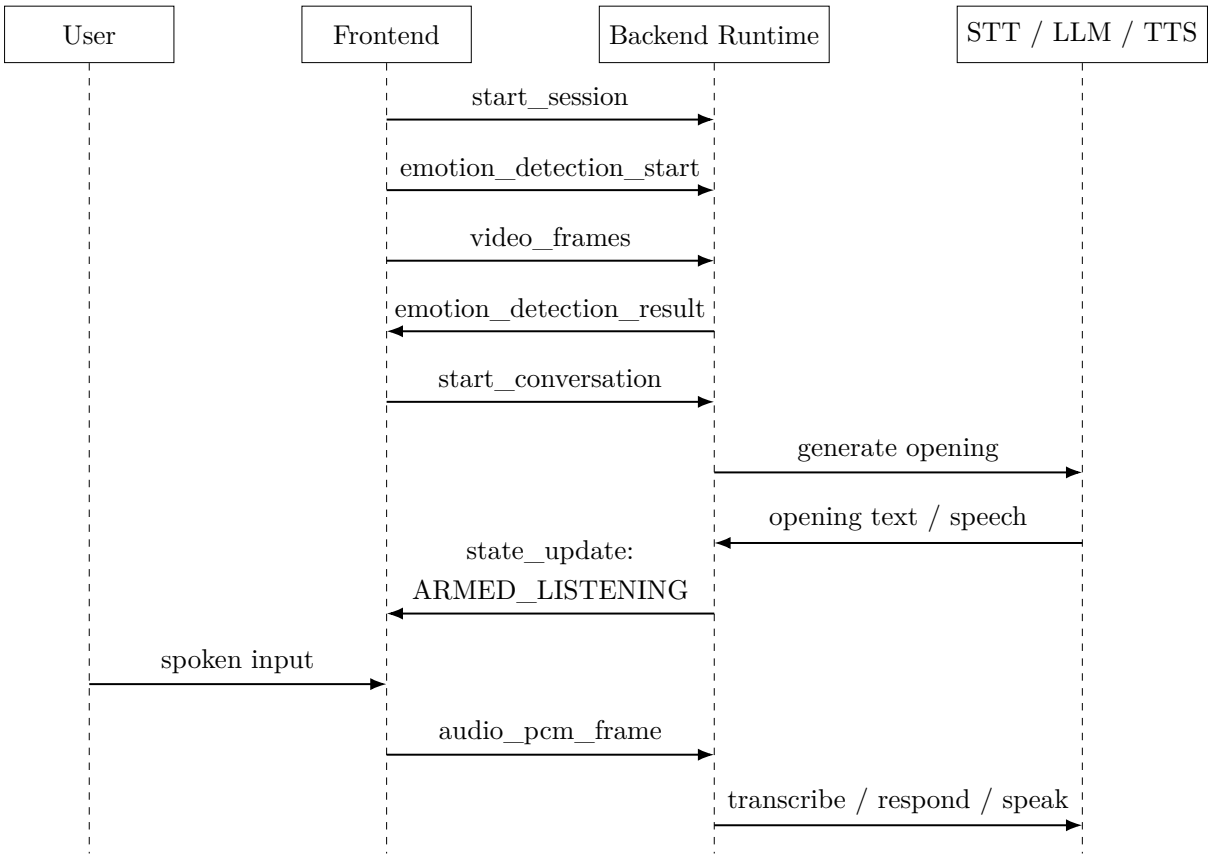


Figure A.3: End-to-end conversation interaction sequence in MoodBridge.

A.4 Emotion Bootstrap Sequence

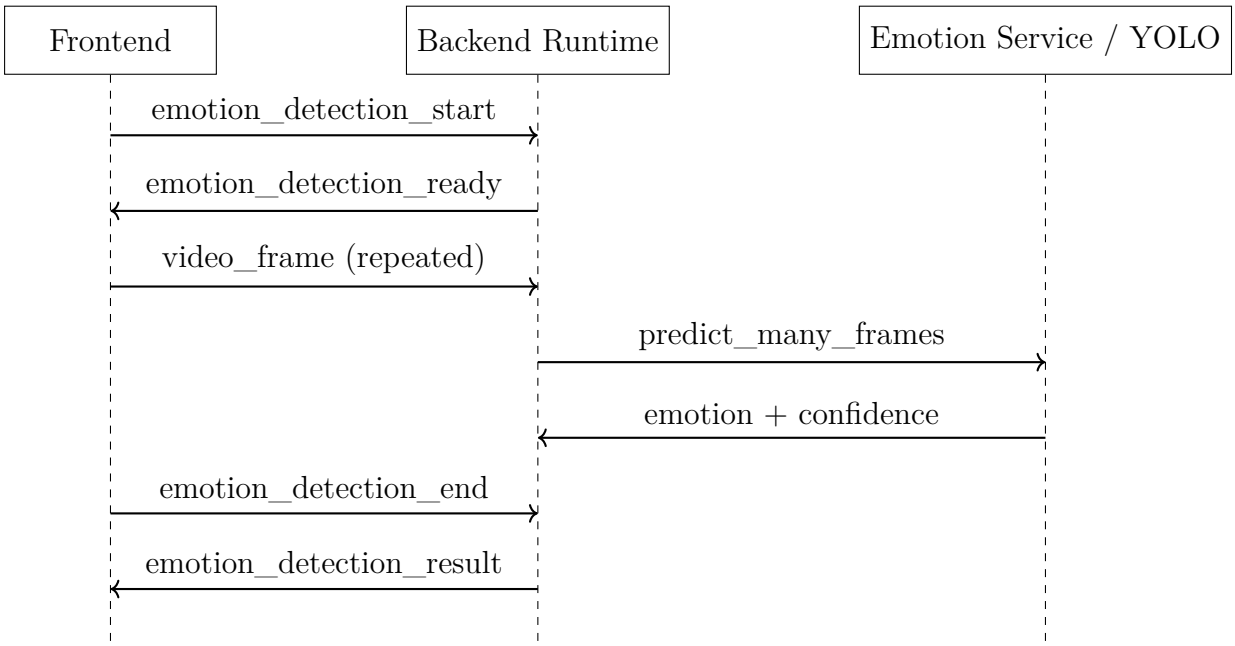


Figure A.4: Emotion bootstrap sequence used before the conversation begins.

A.5 WebSocket Event Flow

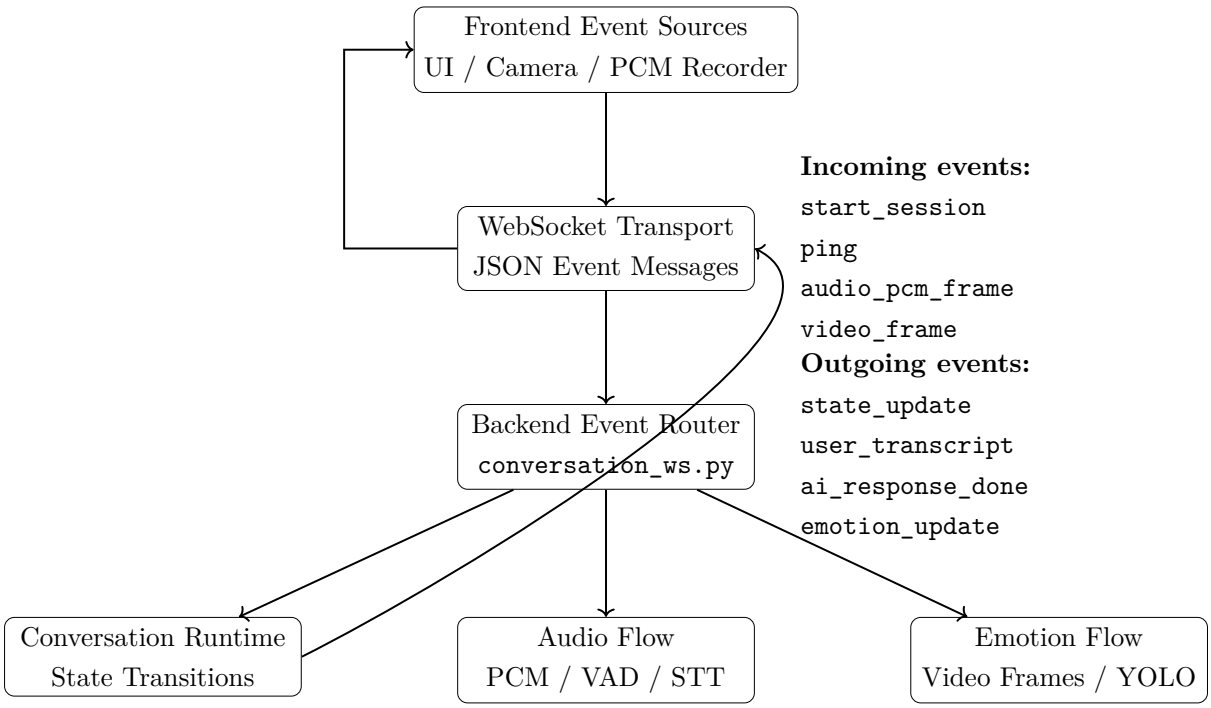


Figure A.5: Event-driven WebSocket flow between frontend and backend runtime components.

A.6 Speech Processing Pipeline

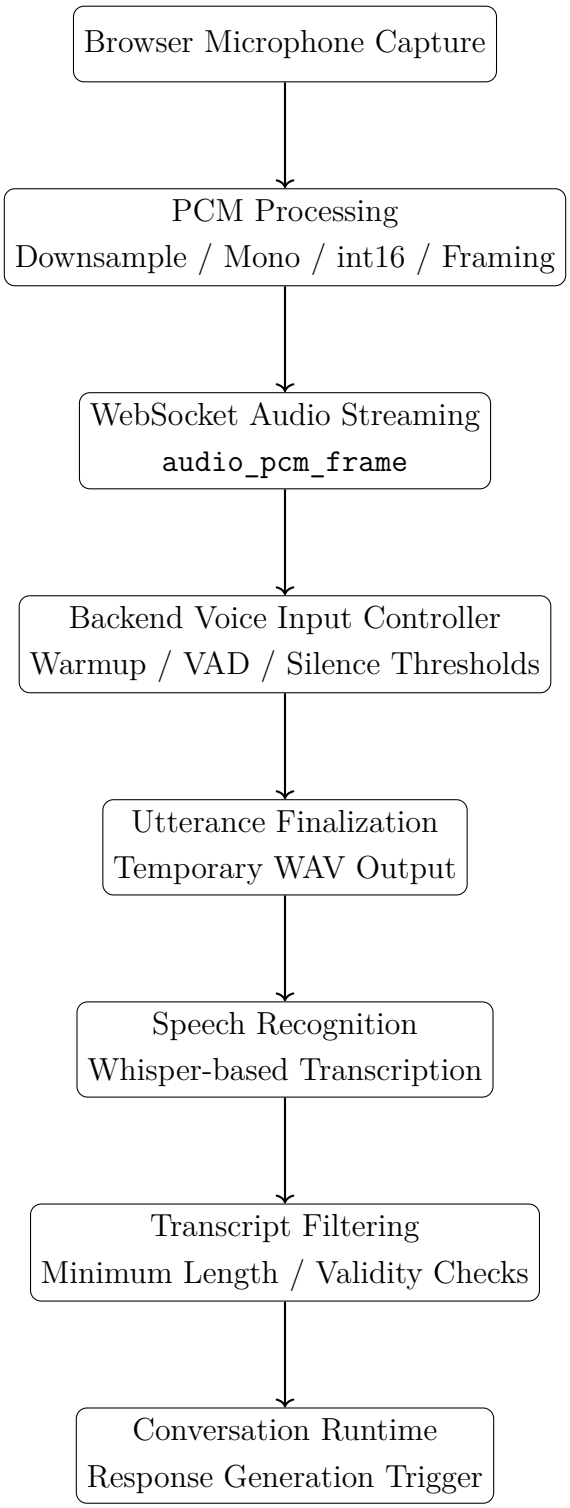


Figure A.6: Speech processing pipeline from browser microphone capture to backend response generation.

A.7 Emotion Detection Pipeline

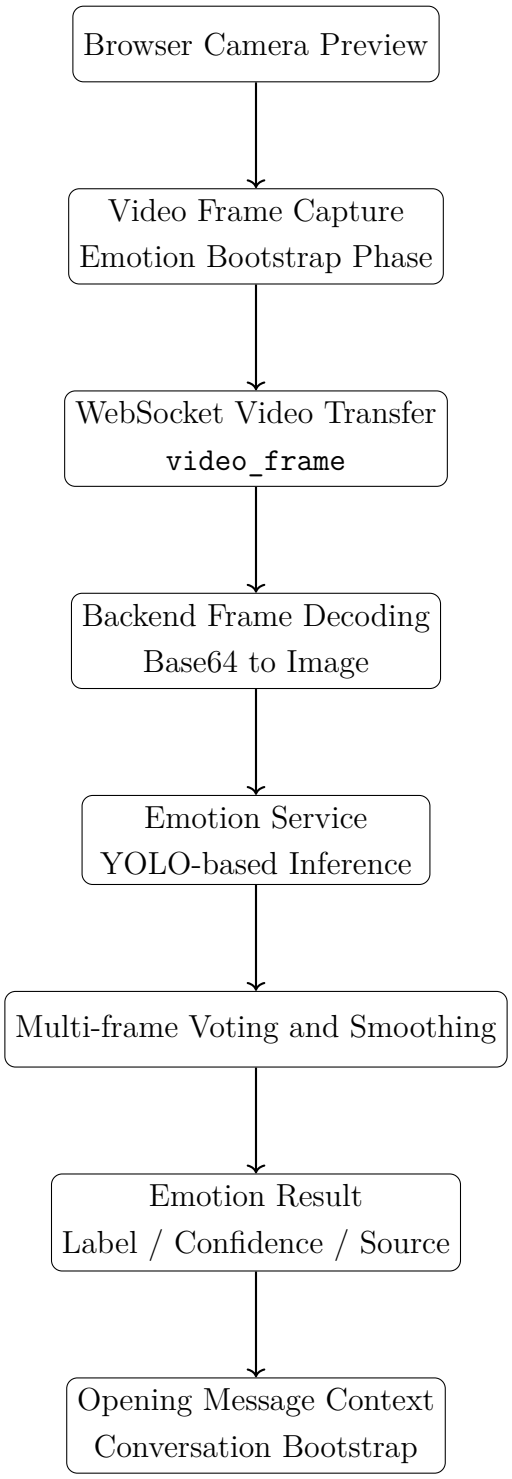


Figure A.7: Emotion detection pipeline used to provide conversational context at session start.

A.8 TTS Response Pipeline

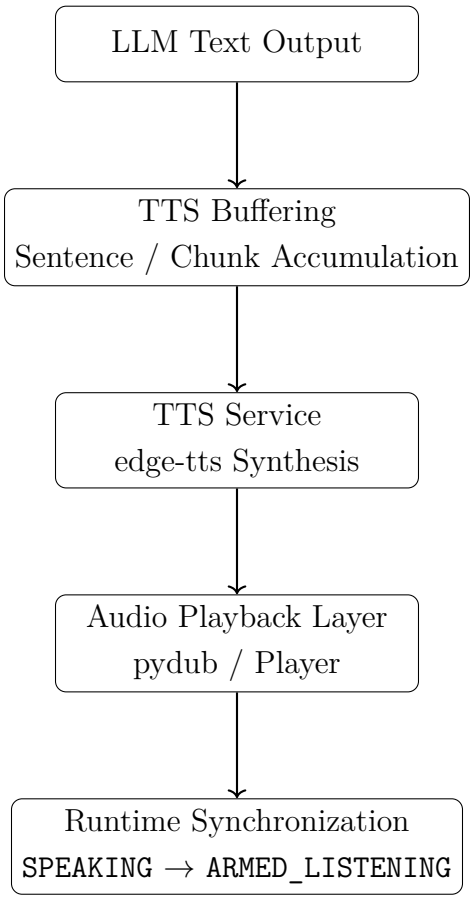


Figure A.8: Text-to-speech response pipeline in MoodBridge.

A.9 Session Lifecycle Diagram

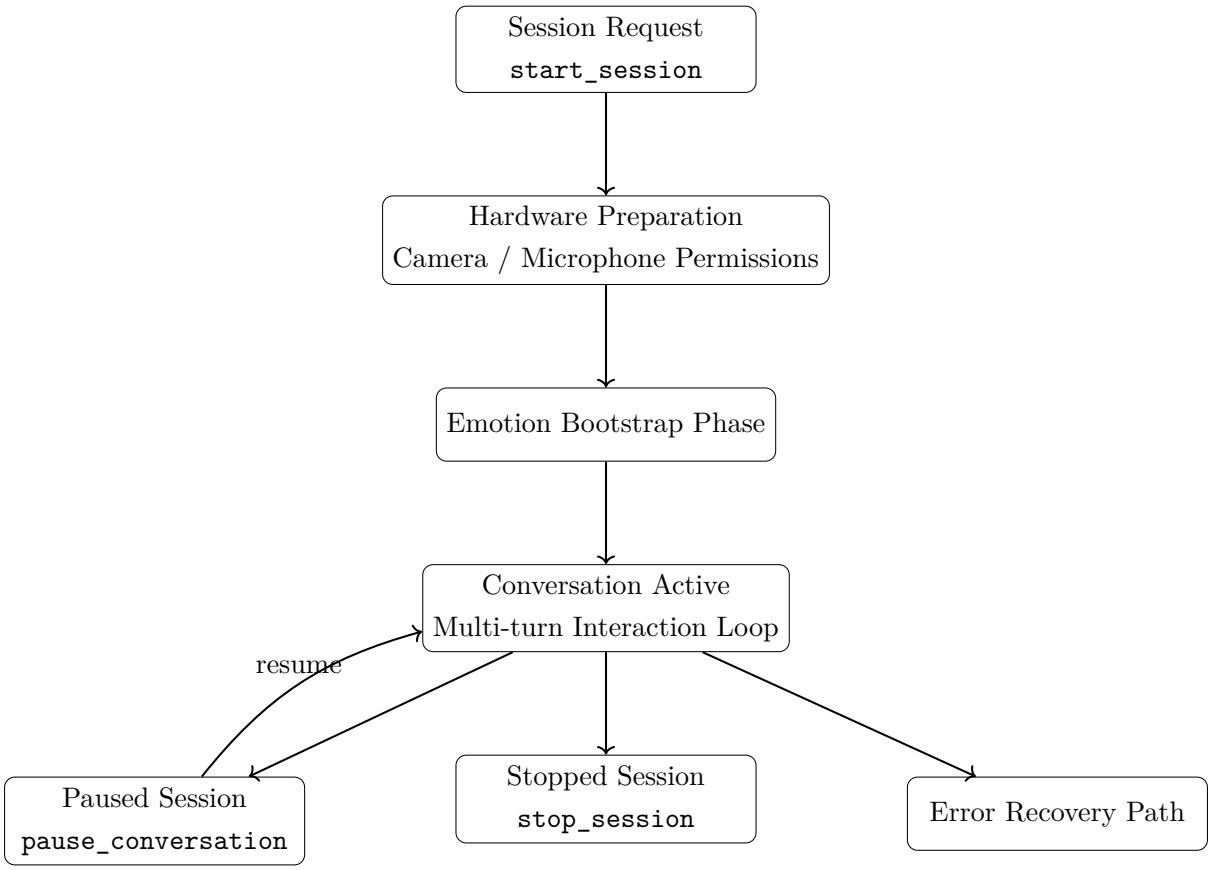


Figure A.9: High-level session lifecycle in MoodBridge.

A.10 Frontend UI Component Structure

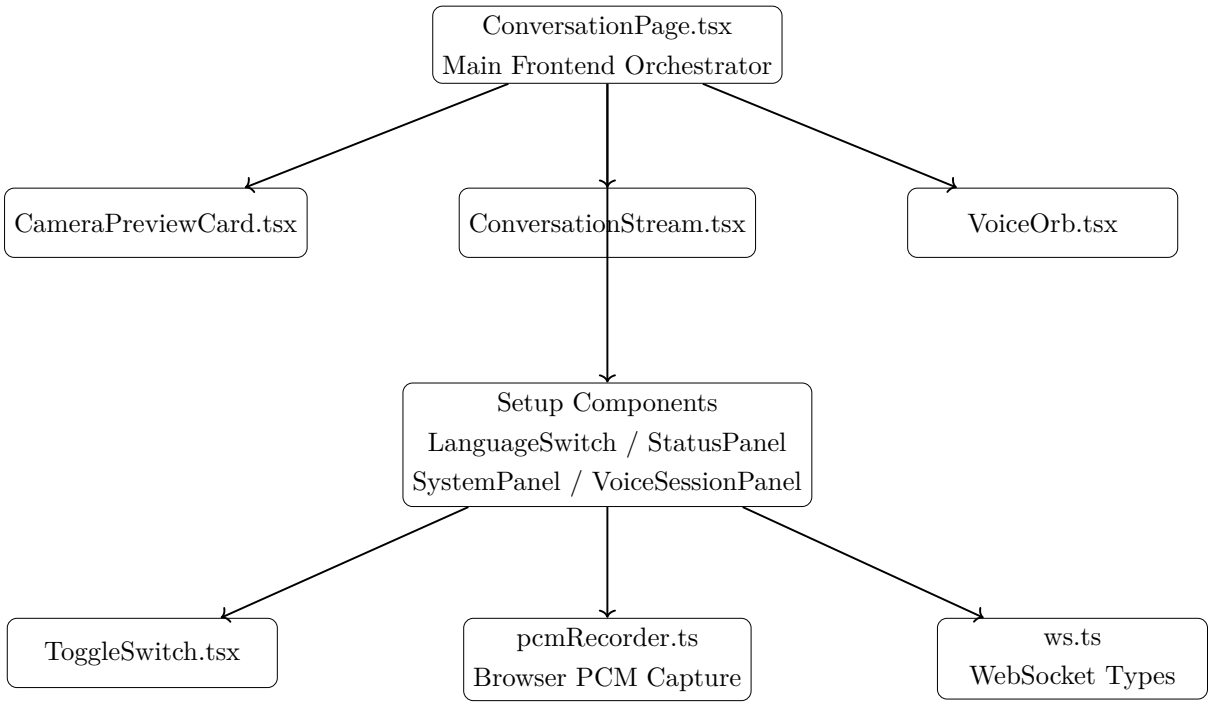


Figure A.10: Frontend UI component structure in MoodBridge.

A.11 Service Dependency Diagram

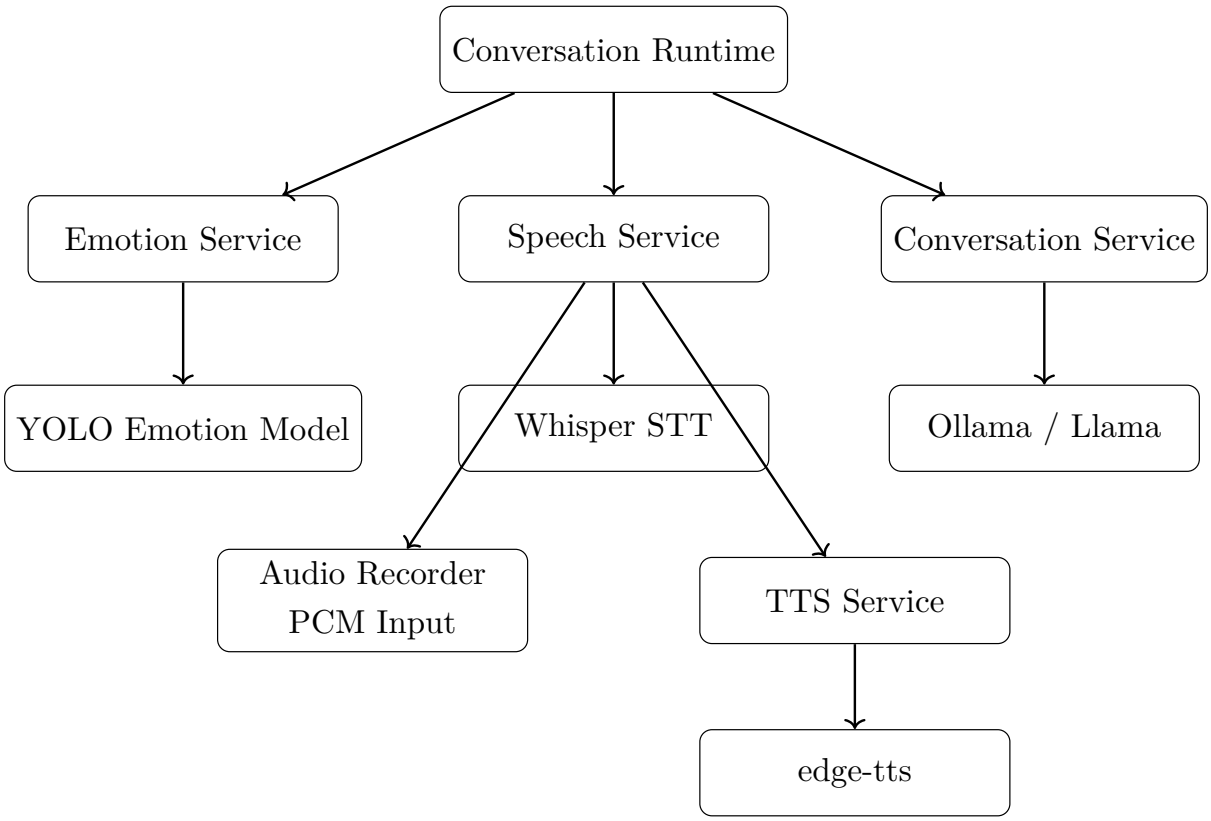


Figure A.11: Dependency relationships between runtime services and underlying models in MoodBridge.

A.12 Runtime Error Handling Flow

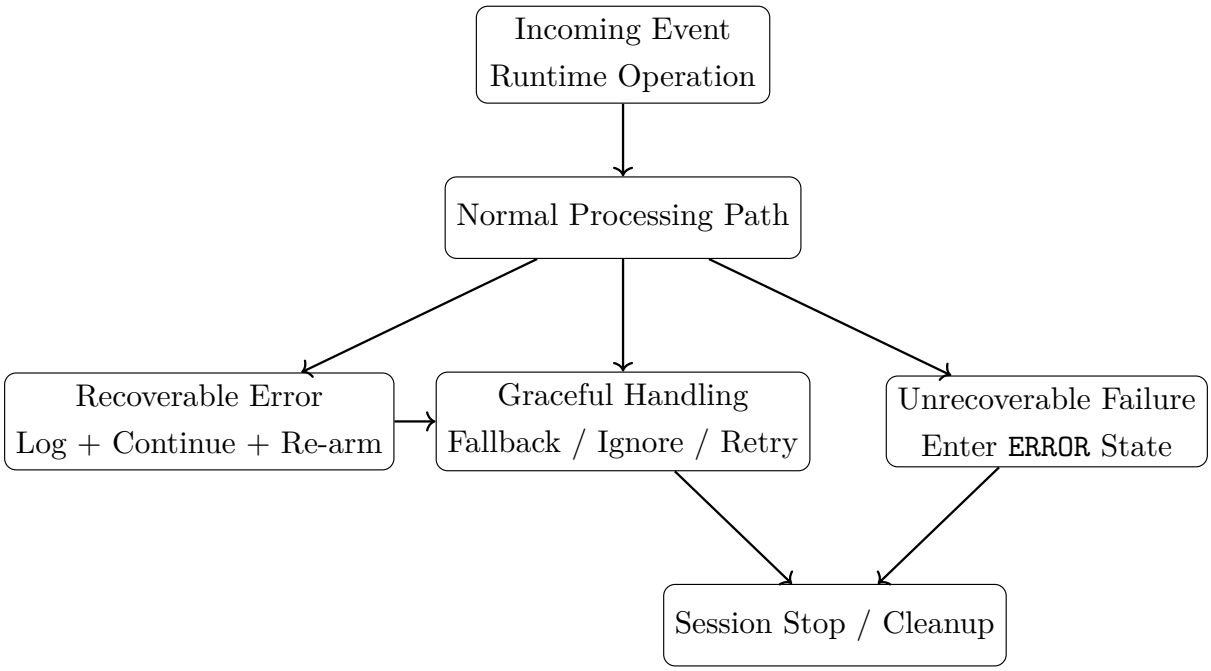


Figure A.12: Simplified runtime error handling flow in MoodBridge.

A.13 Microphone Initialization Timing Fix

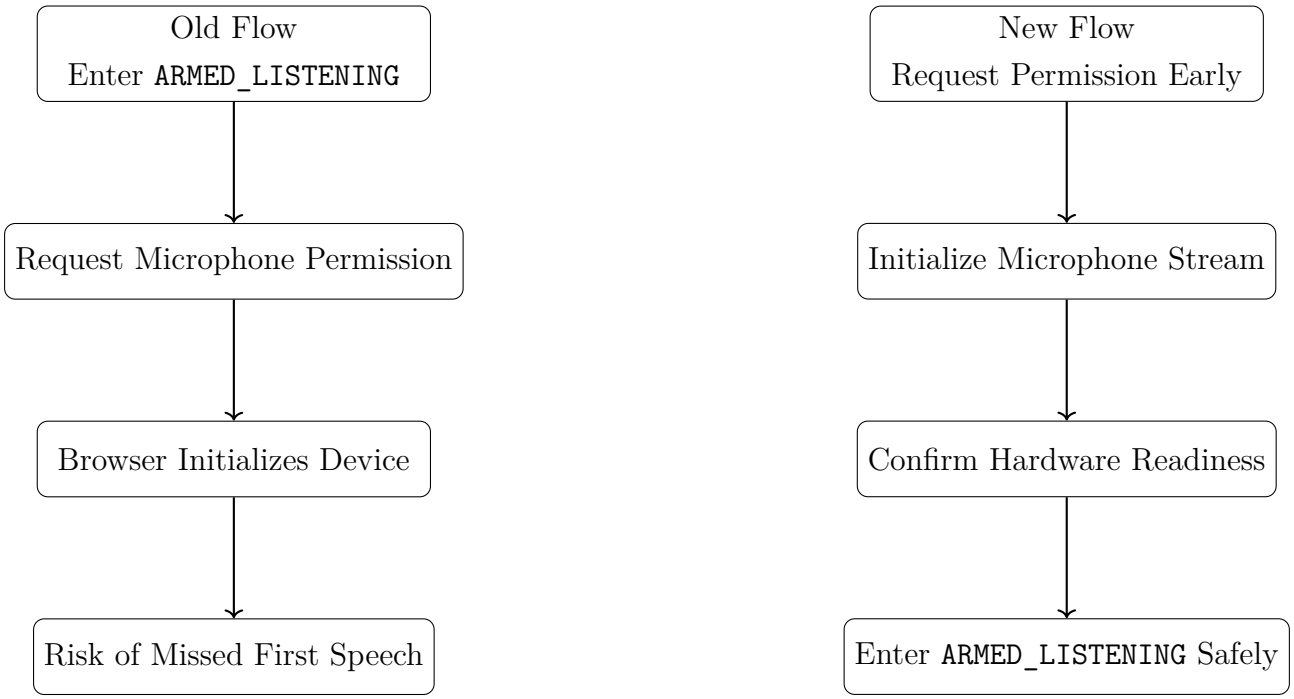


Figure A.13: Comparison between the original and corrected microphone initialization flow.

Appendix B

MoodBridge System Configuration

B.1 Audio Processing Configuration

The realtime audio pipeline operates using fixed timing and sampling parameters to ensure consistent speech segmentation and processing behavior.

Typical runtime configuration values include:

- sample rate: 16000 Hz
- frame duration: 20–30 milliseconds
- audio format: mono PCM `int16`
- silence threshold: configurable energy-based detection
- maximum silence duration: approximately 1 second

These parameters are tuned to balance speech detection accuracy and system responsiveness in realtime environments.

B.2 Network Port Configuration

The system uses the following network ports during operation:

Component	Port	Description
Backend API	8000	FastAPI runtime
Frontend Dev Server	5173	Vite development server
WebSocket Channel	8000	Realtime communication
Local LLM Service	11434	Ollama runtime

Appendix C

MoodBridge Startup Commands

This appendix documents the recommended procedures for launching the MoodBridge system in a reproducible development or demonstration environment. The system consists of two primary runtime components:

- a backend service responsible for realtime conversation orchestration,
- a frontend interface providing browser-based interaction and visualization.

The following sections describe both manual and script-based startup methods, along with required software dependencies and troubleshooting guidance.

C.1 System Prerequisites

Before running the system, ensure that the required runtime dependencies are installed and properly configured.

C.1.1 Required Software

- Python 3.10 or later
- Node.js (recommended version: 18 or later)
- npm (Node Package Manager)
- Conda or Miniconda environment manager
- Web browser with camera and microphone support
- Internet connectivity (required for model downloads or API calls)

C.1.2 Recommended Python Packages

The backend environment typically includes:

- FastAPI
- Uvicorn
- sounddevice
- numpy
- pydub
- edge-tts
- Whisper or compatible speech recognition library
- OpenCV

These dependencies are normally installed automatically through the project environment setup.

C.1.3 Recommended System Utilities

The following utilities are helpful for debugging and runtime validation:

```
pactl
ffmpeg
sox
curl
```

For example, listing available audio output devices:

```
pactl list short sinks
```

—

C.2 Option 1: Manual Startup Using Two Terminals

This method provides the most transparent control over system execution and is recommended during development and debugging.

C.2.1 Terminal 1: Backend Runtime

```
conda activate mood_bridge

source env/llm.sh

cd backend/

uvicorn app.main:app \
  --reload \
  --ws-ping-interval 60 \
  --ws-ping-timeout 60 \
  --timeout-keep-alive 120
```

C.2.2 Terminal 2: Frontend Interface

```
conda activate mood_bridge

source env/llm.sh

cd frontend/

npm run dev
```

After successful startup, the frontend development server typically becomes available at:

```
http://localhost:5173
```

—

C.3 Option 2: Script-Based Startup

For convenience and reproducibility, the system provides helper scripts that automatically initialize both backend and frontend components.

C.3.1 Start the Full System

```
Shell: $ ./scripts/run_main.sh
```

This script typically performs the following operations:

- activates the Conda environment,

- loads runtime environment variables,
- starts the backend service,
- launches the frontend development server,
- initializes required background processes.

C.3.2 Stop the System

```
Shell: $ ./scripts/stop_main.sh
```

This command terminates all running processes associated with the MoodBridge runtime.

—

C.4 Demo Conversation Startup

A simplified demo workflow is available for testing the conversation subsystem independently of the full application stack.

```
Shell: $ ./scripts/run_subs.sh
```

This script typically launches:

- the conversation runtime,
- audio streaming components,
- speech recognition pipeline,
- text-to-speech response generation.

It is commonly used during subsystem validation or rapid testing.

—

C.5 Environment Initialization

The system relies on environment configuration scripts to define runtime parameters such as model paths, API endpoints, and device settings.

C.5.1 Load Environment Variables

```
source env/llm.sh
```

Typical configuration values include:

- LLM API endpoint
- model directory paths
- audio device configuration
- runtime flags

—

C.6 Verification Commands

The following commands can be used to verify that the system is running correctly.

C.6.1 Check Backend Availability

```
curl http://localhost:8000
```

C.6.2 Check Active Network Ports

```
ss -tulnp | grep 8000
```

C.6.3 Check Frontend Server

Open a browser and navigate to:

```
http://localhost:5173
```

—

C.7 Common Troubleshooting Scenarios

C.7.1 Frontend Does Not Start

Possible causes:

- Node.js is not installed
- dependencies are missing

- incorrect working directory

  Recommended fix:

```
cd frontend
npm install
npm run dev
```

—

C.7.2 Backend Fails to Start

Possible causes:

- Conda environment not activated
- missing Python dependencies
- port conflict

  Recommended fix:

```
conda activate mood_bridge
pip install -r requirements.txt
```

—

C.7.3 Microphone or Speaker Not Detected

Use the diagnostic tool:

```
python tools/test_audio_devices.py
```

Or list audio devices:

```
pactl list short sources
pactl list short sinks
```

—

C.8 Engineering Recommendation

For reliable system operation during demonstrations or evaluation, the following practices are recommended:

- always activate the correct Conda environment before startup,
- verify microphone and speaker availability before launching the system,
- run backend and frontend in separate terminals during debugging,
- use script-based startup for repeatable demonstrations,
- monitor runtime logs for connection or device errors.

These practices significantly reduce startup failures and improve runtime stability in realtime conversational systems.

Appendix D

Hardware and Software Requirements

D.1 Minimum Hardware Requirements

- CPU: modern multi-core processor
- RAM: 8 GB minimum
- Storage: 5 GB available disk space
- Microphone: required
- Webcam: required

D.2 Recommended Hardware

- CPU: 6-core or higher
- RAM: 16 GB or more
- GPU: optional but recommended for model inference
- SSD storage

D.3 Software Requirements

- Operating system: Ubuntu Linux
- Python: version 3.10 or later
- Browser: Chrome, Edge, or equivalent modern browser
- Conda: environment management
- Node.js and npm: frontend dependency management

Appendix E

Diagnostic and Development Tools

During development of the MoodBridge system, several standalone utility scripts were created to assist with debugging, hardware verification, and subsystem validation. These tools provide quick diagnostic capabilities independent of the full runtime, allowing developers to isolate issues related to audio devices, text-to-speech output, and language model connectivity.

The scripts are located in the following directory:

```
moodbridge/tools/
    test_audio_devices.py
    test_edge_tts.py
    test_llm_api.py
```

These utilities are intentionally lightweight and can be executed directly from the command line to validate individual components before running the complete system.

E.1 Audio Device Diagnostic Tool

E.1.1 Script

`test_audio_devices.py`

E.1.2 Purpose

This tool is used to inspect the current audio hardware configuration and verify that the correct input and output devices are available to the system. It is particularly useful when diagnosing microphone or speaker selection issues, device enumeration failures, or unexpected audio routing behavior.

E.1.3 Functionality

The script performs the following checks:

- lists all detected audio input and output devices,
- identifies the system default audio device,
- attempts to locate a USB microphone device,
- prints device identifiers and configuration details.

Typical output includes:

- full device list,
- default device index,
- detected external microphone device,
- device name and channel configuration.

E.1.4 Typical Usage

```
python tools/test_audio_devices.py
```

E.1.5 Future Improvement

The tool is planned to be extended to support explicit output device selection. For example, integration with system-level audio utilities such as:

```
pactl list short sinks
```

will allow developers to:

- enumerate available audio output devices,
- identify headphone or speaker sinks,
- programmatically select the preferred output device,
- validate runtime audio routing behavior.

This capability is particularly important when deploying the system on machines with multiple audio interfaces.

—

E.2 Text-to-Speech Verification Tool

E.2.1 Script

`test_edge_tts.py`

E.2.2 Purpose

This tool verifies that the text-to-speech subsystem is functioning correctly. It generates a short speech sample from a test input string using a selected voice and saves the result as an audio file.

The script allows developers to confirm:

- successful connection to the TTS engine,
- correct voice configuration,
- proper audio file generation,
- playback compatibility.

E.2.3 Functionality

The script performs the following steps:

1. accepts a test text string,
2. selects a specified voice,
3. sends the text to the TTS engine,
4. generates an output audio file in MP3 format,
5. optionally plays the generated audio.

E.2.4 Typical Usage

```
python tools/test_edge_tts.py
```

This command produces an output file such as:

```
test_output.mp3
```

E.2.5 Listing Available Voices

The script documentation includes a command-line instruction to list supported voices. This is useful when selecting a voice compatible with the system language or regional settings.

Example:

```
edge-tts --list-voices
```

This command prints all available voices, including language, gender, and voice name.

—

E.3 Language Model Connectivity Test Tool

E.3.1 Script

`test_llm_api.py`

E.3.2 Purpose

This tool verifies connectivity between the backend system and the language model service. It is used to confirm that the LLM API endpoint is reachable and capable of generating responses.

The tool is especially useful when diagnosing:

- network connectivity issues,
- API authentication failures,
- model availability problems,
- response latency anomalies.

E.3.3 Functionality

The script performs the following operations:

1. sends a test prompt to the configured LLM endpoint,
2. waits for a generated response,
3. prints the returned text to the console,
4. reports connection errors if the request fails.

E.3.4 Typical Usage

```
python tools/test_llm_api.py
```

Successful execution produces output similar to:

```
Connected to LLM service
Response received
Hello, how can I help you today?
```

—

E.4 Engineering Role of Diagnostic Tools

These utility scripts play an important role in maintaining system reliability and development efficiency. Instead of debugging the full realtime pipeline, developers can validate individual subsystems independently.

This modular diagnostic approach supports:

- faster root cause identification,
- safer incremental testing,
- reduced debugging complexity,
- improved deployment readiness.

In complex realtime systems such as MoodBridge, small standalone diagnostic tools are often as important as the main application itself.

Appendix F

Project Documentation and Supporting Directories

This appendix summarizes the main supporting directories in the MoodBridge repository and indicates where readers can find supplementary project materials, including diagrams, configuration assets, demonstration scripts, and model files.

The purpose of this appendix is to make the repository easier to navigate for future developers, evaluators, or collaborators who want to inspect the system beyond the main technical report.

F.1 Repository-Level Supporting Materials

In addition to the main runtime components, the repository contains several supporting directories that store reference materials, visual assets, scripts, and subsystem demonstrations. These directories are useful for understanding the evolution of the project, reproducing experiments, and locating technical artifacts related to system design.

F.2 Project Documentation Resources

Readers who want to locate additional design materials, diagrams, or project notes should refer primarily to the `docs/` directory.

F.2.1 `docs/`

The `docs/` directory stores project-level documentation and reference materials generated during development. This folder may include technical notes, architecture descriptions, workflow explanations, and visual system assets.

F.2.2 `docs/diagram/mermaid/`

This directory contains Mermaid-based diagram sources used to describe system architecture, event flow, conversation sequence, and other structural views of the MoodBridge runtime. These files are useful when readers want editable, text-based diagram definitions that can be re-rendered or adapted later.

F.2.3 `docs/diagram/ASCII/`

This directory contains ASCII-based diagrams and lightweight textual visualizations of the system. These representations are helpful for quick inspection, terminal-based documentation, or environments where richer rendering tools are unavailable.

Together, these documentation directories provide a bridge between the written technical report and the underlying engineering artifacts of the project.

F.3 Supporting Directory Descriptions

The following folders are especially important for understanding the broader project structure outside the main backend and frontend application code.

F.3.1 `demos/`

The `demos/` directory contains standalone demonstration scripts used to test or present individual subsystems of MoodBridge without running the full browser-based application.

Typical examples include:

- conversation demo scripts,
- speech input testing,
- webcam and emotion detection testing.

These scripts are useful for subsystem validation, quick experiments, and incremental debugging during development.

F.3.2 `docs/`

The `docs/` directory contains supporting written materials, technical notes, and diagram sources. It acts as the main location for project-level documentation outside the primary report source.

Readers looking for additional reference materials, architecture notes, or editable diagrams should begin here.

F.3.3 `env/`

The `env/` directory contains environment-related helper files used to configure runtime variables and model settings. For example, shell scripts in this folder may define local language model endpoints, API variables, or other startup configuration parameters needed before launching the backend.

This directory supports reproducible runtime setup and helps separate environment configuration from application logic.

F.3.4 `images/`

The `images/` directory stores image assets related to the project. These may include screenshots, reference figures, visual examples, exported diagrams, or other image-based materials used in documentation and presentation.

For readers of the technical report, this folder is a likely location for supporting visual materials referenced during development.

F.3.5 `models/`

The `models/` directory stores model-related resources used by the system, including trained weights and metadata files.

Typical contents include:

- YOLO-based emotion detection model weights,
- class metadata files such as `data.yaml`.

This folder represents the AI model layer of the project and is central to the emotion detection subsystem.

F.3.6 `scripts/`

The `scripts/` directory contains helper shell scripts used to start, stop, or simplify system execution. These scripts support reproducible launches for demonstrations and reduce the need for repeated manual command entry.

Typical usage includes:

- starting the main system,
- launching subsystem demos,
- stopping running processes cleanly.

This folder improves operational convenience and supports consistent testing workflows.

F.4 How Readers Can Use These Materials

A reader who wants to explore the MoodBridge project beyond the main report can use the repository structure in the following way:

- consult `docs/` for supplementary design notes and reference materials,
- inspect `docs/diagram/mermaid/` and `docs/diagram/ASCII/` for editable diagram sources,
- use `demos/` to run isolated subsystem tests,
- review `models/` to understand the model assets used in the emotion detection pipeline,
- use `scripts/` and `env/` to reproduce the runtime setup described in the report,
- check `images/` for project screenshots or supporting visual material.

This organization makes the project easier to inspect, reproduce, and extend, while also preserving a clear separation between application code, model assets, supporting documentation, and operational tooling.

Glossary

YOLO (You Only Look Once) A real-time object detection model based on deep convolutional neural networks. YOLO processes an image in a single forward pass to identify objects and predict their bounding boxes and class probabilities. In the MoodBridge system, a YOLO-based model is used to detect facial expressions and infer the user's emotional state during the emotion bootstrap phase.

Computer Vision (CV) A field of artificial intelligence focused on enabling machines to interpret and analyze visual information from images or video. Computer vision techniques are used in the MoodBridge system to process webcam frames and extract facial features for emotion detection and interaction context generation.

Large Language Model (LLM) A neural network model capable of generating natural language responses based on conversational context.

Speech-to-Text (STT) The process of converting spoken audio into textual representation.

Text-to-Speech (TTS) The process of synthesizing spoken audio from text.

Voice Activity Detection (VAD) A signal processing technique used to detect when a user starts and stops speaking based on audio energy patterns.

PCM Audio Pulse Code Modulation audio format used for low-level audio streaming with deterministic timing.

State Machine A structured control model that defines system behavior as a set of discrete states and transitions between them.

Session Lifecycle The sequence of states that a conversation session passes through, including initialization, active interaction, pause, and termination.

Runtime The period during which a program is actively executing and managing system operations.

Service Layer A modular architectural layer that encapsulates functional subsystems such as speech processing, emotion detection, and conversation management.

Interrupt Handling A runtime mechanism that allows the system to safely stop or redirect ongoing operations when a new input event occurs.

Barge-in A capability that allows a user to interrupt the assistant while it is speaking.

Streaming A data processing approach in which information is transmitted and processed incrementally in small chunks rather than as a complete batch.

Latency The time delay between an input event and the corresponding system response. In realtime conversational systems, latency directly affects interaction smoothness and user experience.

Cold Start The initial delay that occurs when a system component, such as a language model, is loaded into memory for the first time.

Timeout A predefined time limit after which an operation is automatically terminated if no response or activity is detected.

Fallback A predefined alternative action executed when the primary operation fails.

WebSocket A persistent communication protocol that enables bidirectional realtime data exchange between the browser and backend server.

WebSocket Session A persistent communication context established between the frontend and backend that maintains conversation state and enables continuous realtime message exchange.

Heartbeat A periodic signal used to verify that a connection or component remains active and responsive.

Keepalive A mechanism that periodically sends signals to ensure that a connection remains active and to prevent unintended disconnection.

Transport Layer The communication layer responsible for transmitting data between system components, such as a WebSocket channel.

Realtime System A system that processes input and produces output within predictable time constraints.

Emotion Bootstrap A short initialization phase in which the system captures camera input to estimate the user's emotional state before starting a conversation.